



ORBIT Advanced Technologies

ISRAEL: P.O.B. 3171, Industrial Zone, Netanya 42131 - Tel.: 972-9-8605777 - Fax: 972-9-8621701

U.S.A.: 506 Prudential Rd., P.O.B. 999, Horsham, PA. 19044 - Tel.: 215-674-4220 - Fax: 215-674-5108

axis 1: "plate"

COMMUNICATION LINK

for

AL-4706-3A

with a list of adaptations for

AL-4806-3A

AL-4906-3A

AL-4901-3A

PROPRIETARY DATA

Information contained herein is the sole property of ORBIT Advanced Technologies and is not for publication and/or duplication; and it may not be passed to any other party without written authorization from ORBIT Advanced Technologies.

June 1997

Revised by:

HARVEY ROSEN

Checked by:

ZVIKA ZUKER

Rev. I

For AL-4706-3A firmware version 1.7 or later
For AL-4806-3A firmware version 1.3 or later
For AL-4906-3A firmware version 1.3 or later
For AL-4901-3A firmware version 1.3 or later



Note for users of :

AL-4806-3A

✓ **AL-4906-3A**

AL-4901-3A

The communication link for these units is slightly different from that of the AL-4706-3A.

The differences are listed in appendix B.

Prior to using this manual, **read the applicable part of appendix B.**



TABLE OF CONTENTS

	<u>Page</u>
PREFACE	1
MANUAL OUTLINE	2
CHAPTER 1 - GENERAL DESCRIPTION	3
1.1 Scope	3
1.2 Hardware Options	3
1.3 RS-232-C Specifications	3
1.4 IEEE-488 Specifications	4
1.5 Data Framing	4
CHAPTER 2 - ENTERING DATA FROM CONTROLLER	5
2.1 IEEE-488 (GPIB)	5
2.1.1 Parallel Poll	7
2.1.2 Serial Poll	14
2.1.3 PPOLL or SPOLL?	16
2.1.4 Service Request Interrupt	17
2.1.5 Increment Values Format	19
2.2 RS-232-C	20
2.3 Controller Output Buffers	21

TABLE OF CONTENTS (Cont'd)

	<u>Page</u>
CHAPTER 3 - COMMUNICATION LINK COMMANDS	25
3.1 Scope	25
3.2 Main Commands and their Hierarchy	26
3.3 "LOAD" Parameters	32
3.3.1 "P" - Boundary Position	32
3.3.2 "A" - Axis Selection	34
3.3.3 "V" - Velocity	38
3.3.4 "D" - Direction	42
3.3.5 "M" - Mode of Motion	44
3.3.6 "J" - Step Value (in Raster Mode) or Number of Scans (in Sector Mode) ..	47
3.3.7 "I" - Increment Step in RASTER (or SECTOR) Mode	48
3.3.8 "F" - Bus Increments Enable/Disable	50
3.3.9 "C" - PPOLL Configure	51
3.3.10 Configuration Parameters	52
3.3.10.1 "O" - Offset	53
3.3.10.2 "B" Software Limits	54
3.3.10.3 "W" Lock Window	55
3.3.10.4 "c" - Gain Factor	56
3.3.10.5 "g" - ACC/DEC (Acceleration/Deceleration) Factor	57
3.3.10.6 "n" - Type	58
3.3.11 "i" - Baud-Rate Parameter for RS-232-C Communication (Option 6)	59
3.3.12 "j" - DC-OUT Range Parameter (Option 6)	60
3.4 "READ" Parameters	61
3.4.1 "P" - Read Predefined Boundary Position	62
3.4.2 "A" - Read Axis Selection	64
3.4.2.1 For one-axis-at-a-time Controllers	64
3.4.2.2 For two-axes-at-a-time Controllers	66
3.4.3 "V" - Read Predefined Velocity	68
3.4.3.1 For one-axis-at-a-time Controllers	68
3.4.3.2 For two-axes-at-a-time Controllers	69
3.4.4 "D" - Read Predefined Direction	70
3.4.5 "J" - Read Step Value (in Raster mode), or Number of Scans (in Sector mode)	71
3.4.6 "I" - Read Predefined Increment Step	72

TABLE OF CONTENTS (Cont'd)

	<u>Page</u>
3.4.7 Read Configuration Parameters	73
3.4.7.1 "O" - Read Predefined Offset	74
3.4.7.2 "B" - Read Predefined Software Limits	75
3.4.7.3 "W" - Read Predefined Lock Window	76
3.4.7.4 "c" - Read Predefined Gain	77
3.4.7.5 "g" - Read Predefined ACC/DEC Factor	78
3.4.7.6 "n" - Read Predefined Type	79
3.4.8 "i" - Read Predefined Baud-Rate Parameter for RS-232-C Communication	80
3.4.9 "j" - Read Predefined DC-OUT Range Parameter	81
3.5 "RUN" State Commands	82
3.6 Current Status Query Commands	83
3.6.1 For one-axis-at-a-time Controllers	83
3.6.1.1 Response to "X1<"	86
3.6.1.2 Response to "X2<"	87
3.6.1.3 Response to "X3<"	88
3.6.1.4 Response to "X4<"	89
3.6.2 For two-axes-at-a-time Controllers	92
3.6.2.1 Response to "X1<"	95
3.6.2.2 Response to "X2<"	96
3.6.2.3 Response to "X3<"	97
3.6.2.4 Response to "X4<"	98
3.6.2 For two-axes-at-a-time Controllers	92

TABLE OF CONTENTS (Cont'd)

	<u>Page</u>
CHAPTER 4 - "COMM LINK" Commands Sequences	101
4.1 Scope	101
4.2 Chaining the "COMM LINK" Commands	102
4.2.1 Ex. No. 1	103
4.2.2 Ex. No. 2	105
4.2.3 Ex. No. 3	107
4.3 Setting up different motions - LOADING & RUNNING only	109
4.3.1 Setting Up a Track to Target Motion	110
4.3.2 Setting Up a RASTER SCAN (A) Motion	112
4.3.3 Setting Up a RASTER SCAN (B) Motion	114
4.3.4 Setting Up a SECTOR SCAN Motion	116
4.3.5 Setting Up a SLEW Motion	118
4.4 ENTERING DATA Routines	120
4.4.1 Entering DATA Using PPOLL Method	121
4.4.2 Entering DATA Using SPOLL Method	125
4.4.3 Entering DATA Using SRQ INTERRUPT Method ...	129
4.5 Setting Up Different Motions - Loading, Verification & Monitoring	133
4.5.1 Setting Up a Track to Target Motion	134
4.5.2 Setting Up a RASTER SCAN Motion	140
4.5.3 Setting Up a SECTOR SCAN Motion	147
4.5.4 Setting Up a SLEW Motion	154
4.6 Simple Applications	160
4.6.1 Reading GPIB Increments using PPOLL	160
4.6.2 Reading GPIB Increments using SRQ Interrupt	162
4.6.3 Stepping Positioner using "TRACK" Mode	165
4.6.4 Stepping Positioner using "DIRECT" Mode ...	168
4.6.5 Counting Increments by SRQ using SPOLL	171
4.6.6 Counting Increments by SRQ using Interrupt	173



TABLE OF CONTENTS (Cont'd)

	<u>Page</u>
APPENDIX A - "MAIN" (V1.3) PROGRAM DOCUMENTATION	175
A.1 GENERAL	176
A.2 PURPOSE	176
A.3 OPERATION INSTRUCTIONS	177
A.4 HOW DOES IT WORK?	178
APPENDIX B - LIST OF ADAPTATIONS FOR: AL-4806-3A/AL-4906-3A/AL-4901-3A CONTROLLERS	180
B.1 AL-4806-3A	180
B.2 AL-4906-3A	183
B.3 AL-4901-3A	185

LIST OF TABLES

		<u>Page</u>
TABLE 2-1	PPOLL Byte	8
TABLE 2-2	SPOLL Byte	10
TABLE 2-3	SRQ Byte	10
TABLE 2-4	MESSAGE TYPE	10
TABLE 2-5	EVENT MESSAGES	11
TABLE 3-1	MAIN COMMANDS	26
TABLE 3-2	CURRENT STATUS COMMANDS	72

LIST OF FIGURES

FIGURE 2-1	IEEE-488 ADDRESS DIP-SWITCH	6
FIGURE 2-2	PARALLEL POLL ROUTINE	9
FIGURE 2-3	"INVESTIGATE" ROUTINE	13
FIGURE 2-4	SERIAL POLL ROUTINE	15
FIGURE 2-5	SRQ INTERRUPT SERVICE ROUTINE	18
FIGURE 3-1	MAIN COMMANDS HIERARCHY	27
FIGURE 3-2	"PROPER ORDER OF WORK" (WITHOUT MONITORING)	31
FIGURE 3-3	"PROPER ORDER OF WORK" (WITH MONITORING)	85
FIGURE 3-4	"PROPER ORDER OF WORK" (WITH MONITORING)	94



PREFACE

This is a "COMMUNICATION LINK" manual. In order to understand it better, we must first define the "COMM. LINK":

The "COMM. LINK" is a set of commands, which are accepted by the controller via a communication BUS: IEEE-488 or RS-232-C.

This set of commands ("COMM. LINK COMMANDS") must not be confused with the IEEE-488 (or RS-232-C) BUS standard commands and controls. After all, from IEEE-488 or RS-232-C BUS "Point of View" the COMM LINK COMMANDS are just strings of DATA bytes (mostly ASCII characters)! The CONTROLLER, however, is able to interpret these DATA bytes as different commands for programming modes of motion, motion control or interrogation and measuring of the positioner system.

Therefore, a logical conclusion is, that the "COMM LINK" is actually, for the experienced programmer, the essential command syntax contained in chapter 3. Other chapters contain auxiliary descriptions, explanations and examples.

It is important to emphasize that this manual does not replace the OPERATION MANUAL. Commands described herein activate in the CONTROLLER different functions and processes (such as RASTER, TRACK, INCREMENTS, etc.). The "COMM LINK" does not explain the nature of these functions: this is done in detail in the OPERATION MANUAL.



MANUAL OUTLINE

- CHAPTER 1 - Describes some general points about the communication BUS hardware implementation.
- CHAPTER 2 - Describes how to PPOLL and SPOLL the CONTROLLER. SPOLL and PPOLL are IEEE-488 commands, used to pass data from CONTROLLER to computer, but it is shown herein, how the concept of "SPOLL" is also implemented for RS-232-C.
- CHAPTER 3 - Describes the COMM LINK COMMANDS syntax. No special reference is made as to the use of IEEE-488 or RS-232-C hardware interfaces nor any particular computer. These are usually described in detail in appropriate manuals.
- CHAPTER 4 - Describes some frequently used COMM LINK COMMANDS sequences (such as setting up a RASTER SCAN motion, TRACK motion, etc.).

An explanation of how multiple COMM LINK COMMANDS may be chained together in one string is also included.



CHAPTER 1

GENERAL DESCRIPTION

1.1 Scope

This manual provides the user of ORBIT's AL-4706-3 CONTROLLER with information necessary to interface it to a general purpose computer.

1.2 Hardware Options

The hardware implementation is in two options:

- Serial: RS-232-C
- Parallel: IEEE-488

It is possible to operate both but not simultaneously. Activation of the two is as follows:

- Serial bus is activated after a reset (FRONT PANEL reset or POWER-ON reset)
- Parallel bus is activated automatically whenever the CONTROLLER recognizes activity on this BUS.

1.3 RS-232-C Specifications *wait no not this*

The serial link consists of three wires: Txd, Rxd, GND.

The link features are:

- * Full duplex asynchronous
- * Baud rate: factory set to 9600 bps
may be reprogrammed (see 3.3.11)
- * Communication byte structure: 1 start bit
8 data bits
1 parity bit (even)
1 stop bit



1.4

IEEE-488 Specifications

+ his. Want + his

In general this link is according to the IEEE-488 standard.

The IEEE-488 commands accepted by the CONTROLLER are:

- | | | |
|---------------------------------|--|---------------------------|
| * SDC - Select Device Clear | | Primary Commands Group |
| * DCL - Device Clear | | |
| * SPE - Serial Poll Enable | | |
| * SPD - Serial Poll Disable | | |
| | | |
| * LISTEN/TALK Addresses 0 to 31 | | LISTEN/TALK Address Group |
| * UNL - Unlisten | | |
| * UNT - Untalk | | |

Exception to IEEE standards are:

- 1) The PPOLL CONFIGURE command is implemented indirectly - by means of a COMM LINK COMMAND and not by the IEEE-488 PPC and PPU commands.

1.5

Data Framing

In both cases (RS-232-C and IEEE-488) all the messages between the CONTROLLER and COMPUTER and vice versa are strings of DATA (mostly ASCII characters) terminated by an ASCII sequence "CR", "LF" (DEC 13, 10).

CHAPTER 2ENTERING DATA FROM CONTROLLER2.1 IEEE-488 (GPIB)

Up to 15 devices which communicate with the computer may be connected in parallel to the GPIB bus. They serve as GPIB TALKER/LISTENER devices, whereas, only one computer (or any other specially designed unit) serves as the main GPIB CONTROLLER at any time. Each device has its own unique address used by the GPIB CONTROLLER in the communication process.

In the case of the controller, the device address is hardware-set by a dip switch assembly located on its rear panel (see Figure 2-1).

The assembly consists of eight binary switches each of which may be OPEN (1) or CLOSED (0). Only the five least significant bits are used by the controller for IEEE addressing. The range of legal assignable addresses is 0-30. The user is cautioned against the use of duplicate addresses already assigned to other devices (diskette-drives, printers, etc.).

Note

The configuration for AL-4806-3A, AL-4906-3A and AL-4901-3A is determined by the state of the dip switch No. 8. This switch should be set according to the following table:

Configuration	Dip Switch No. 8
The Controller used as a LOCAL unit with AL-4706-3B	"0"
The Controller used as a stand-alone hooked to a Computer.	"1"

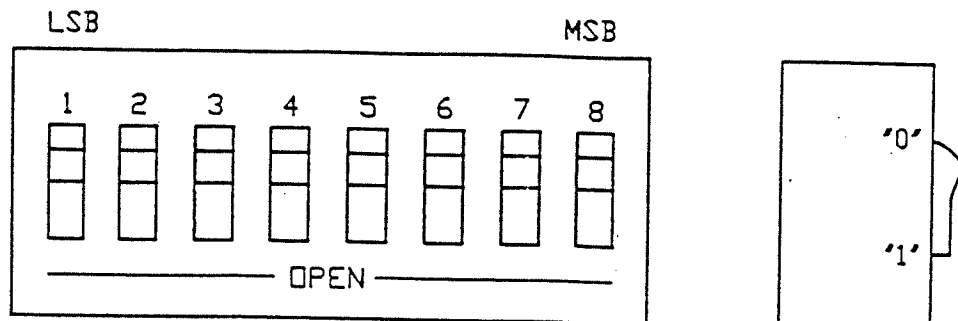
8 7 6 5 4 3 2 1

✓✓✓✓ - ✓✓✓✓

1 0 0 1 - 1 1 1 1

16 8 4 2 1

0 - 30



IEEE-488 ADDRESS

Fig. 2.1: IEEE-488 Comm Link Address Dip-Switch



2.1.1 Parallel Poll



The computer polls the bus to which the devices are connected, in order to check whether a message is pending. The form of the Parallel Poll instruction is:

I should be polling
or Spolling, not
Sending Stb
commands

PARALLEL POLL (or PPOLL Interface No.)

The PPOLL Interface No. depends on the type of computer employed and on the number of buses (GPIB-cards) housed by the computer.

Upon delivery of the PARALLEL POLL instruction, a PARALLEL POLL byte is constructed from bits controlled by the individual devices connected to the bus. Their position and their values convey to the computer:

- (a) if there is a pending message from a device,
- (b) which device has a pending message, and possibly,
- (c) the type of message.

The idea of the PARALLEL POLL instruction is, therefore, to receive, by means of a single polling call, information about all the devices that request service.

In our case, two of the bits of the said PARALLEL POLL byte are dedicated to the controller. The other six bits are used to indicate if a message signal was received from another device. It is worthwhile mentioning that this allocation of two bits to a particular device - in our case the controller - constitutes a slight deviation from the accepted standard of GPIB.



AL-4706-3A COMMUNICATION LINK

The two bits dedicated to the controller indicate the urgency of an awaiting message, and its type. The position of these two bits in the byte is arbitrary, and in fact is determined by a COMM LINK instruction belonging to the LOAD sequence (see sector 3.3.9). For the sake of our present example we will assume that they are bits B3 and B4.

Table 2.1: PPOLL Byte

OTHER DEVICES			MESSAGE	INCREMENT	OTHER DEVICES		
B7	B6	B5	B4	B3	B2	B1	B0

When the INCREMENT bit B3, is set to "1", the controller informs the computer that during a scan, an increment angle has been attained, there is therefore an urgent message (the angle value) to be handled immediately. (A TTL increment pulse may also have been generated).

When the MESSAGE bit B4 is set to 1, the controller is informing the computer that a message (either a response to an inquiry, or a message concerning a random event) is waiting to be transmitted.

In the first case (B3=1), the computer should branch to a routine which services the increment event directly: enters the increment value. In the second case (B4=1), the computer will continue investigating the nature of the message by means of SPOLL.

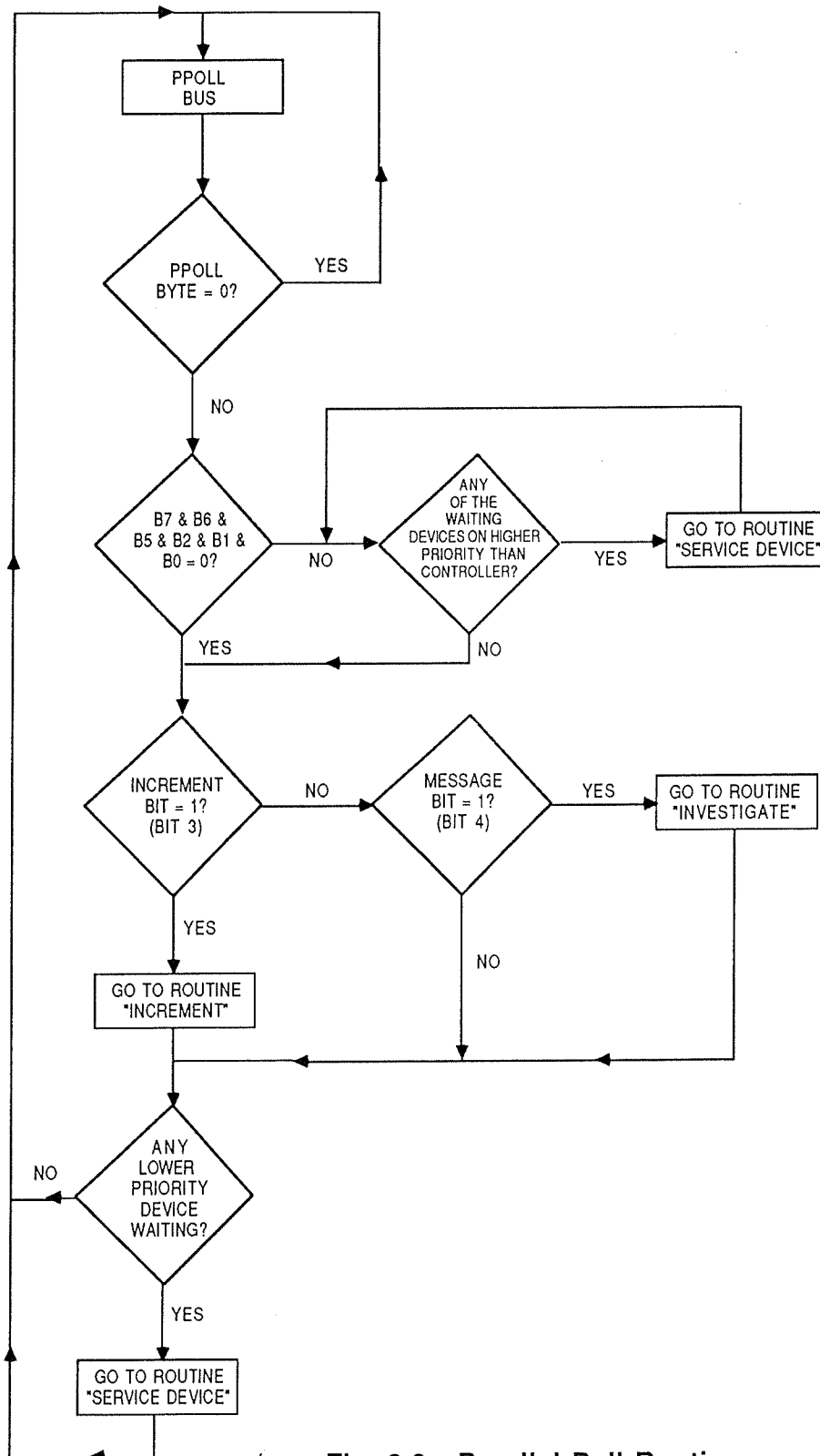


Fig. 2.2: Parallel Poll Routine



The investigation is initiated by another functional instruction:

SERIAL POLL (or SPOLL Device Address)

If the designated device is the controller it returns a byte called Serial Poll Byte (SPB):

Table 2.2: SPOLL Byte

MESSAGE TYPE	SERVICE REQUEST	0	1	EVENT TYPE			
B7	B6	B5	B4	B3	B2	B1	B0

Table 2.3: SRQ bit

B6 = 0	No message
B6 = 1	CONTROLLER message pending

Table 2.4: MESSAGE TYPE

B7 = 0	Event: CONTROLLER message consists of the SPOLL byte only. The information it contains is according to Table 2.5
B7 = 1	CONTROLLER message consists of this SPOLL byte and some more (3 to 20) bytes which must be ENTERED



AL-4706-3A COMMUNICATION LINK

Table 2.5: EVENT MESSAGES

B7 B6 B5 B4	B3 B2 B1 B0	EVENT	SEE NOTES	HEX CODE	DEC CODE
0 1 0 1	0 0 0 0	Both limits on	(1,2)	50	80
0 1 0 1	0 0 0 1	Active axis reached CW limit	(2)	51	81
0 1 0 1	0 0 1 0	Active axis reached CCW limit	(2)	52	82
0 1 0 1	0 0 1 1	Active axis pulled out from limit	(2,3)	53	83
0 1 0 1	0 1 0 0	Illegal data or command	(4)	54	84
0 1 0 1	0 1 0 1	Not used		--	--
0 1 0 1	0 1 1 0	Not used		--	--
0 1 0 1	0 1 1 1	Not used		--	--
0 1 0 1	1 0 0 0	Positioners stopped in a locking window		58	88
0 1 0 1	1 0 0 1	Not used		--	--
0 1 0 1	1 0 1 0	Not used		--	--
0 1 0 1	1 0 1 1	Not used		--	--
0 1 0 1	1 1 0 0	Operating axis crossed an increment angle	(5)	5C	92
0 1 0 1	1 1 0 1	Not used		--	--
0 1 0 1	1 1 1 0	Self-test failed	(6)	5E	95
0 1 0 1	1 1 1 1	Self-test passed	(6)	5F	96



Notes:

- (1) This event will appear whenever switching from a connected axis to an unconnected one.
- (2) It must be noted that the limit messages are output by the CONTROLLER on their occurrence. For more complete limit switching status (Axis A limit or Axis E limit, software limit or hardware limit, etc.) one should use an "X4<" query (see para. 3.6.4).
- (3) This event will appear whenever switching from an unconnected axis (both limits ON!) to a connected axis.
- (4) Commands sent by the computer must confirm with a certain format, and data values must remain within specified limits. If any of these rules is violated, the CONTROLLER issues a message indicating that an illegal command/data has been received.
- (5) If GPIB increments are fully enabled, the SPOLL for an increment update output by the CONTROLLER is 220 (B7 = 1), followed by 6 or 7 ASCII characters of increment position (see para. 2.1.5).
But, if an F|e|< disable command (see para. 3.3.8) was initiated, the SPOLL for an increment will be 92 (B7 = 0), without any following data.
- (6) Self-test is initiated by a special command (see para. 3.2).

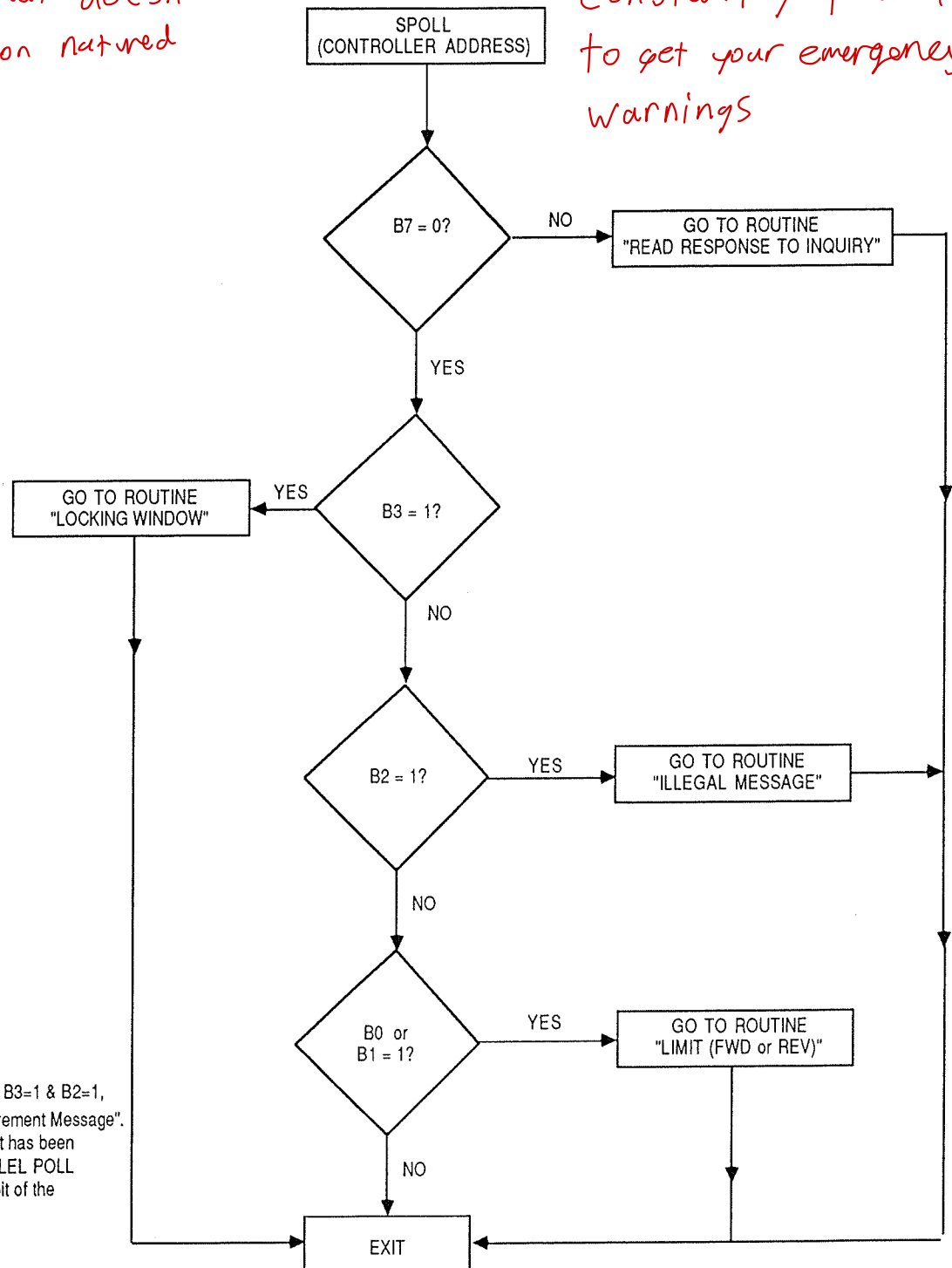
Fig. 2-3 is a flow chart depicting a possible routine for investigating the message type.



AL-4706-3A COMMUNICATION LINK

- need a loop that doesn't
cause a python natured
mess

constantly poll this
to get your emergency
warnings



NOTE:

There is no need to check if B3=1 & B2=1, the meaning of which is "Increment Message". This cannot occur because it has been serviced during the PARALLEL POLL phase when the increment bit of the PARALLEL POLL was 1.

Fig. 2.3: "Investigate" Routine



2.1.2 Serial Poll

increment: angle change
has occurred
found using B2 and B3

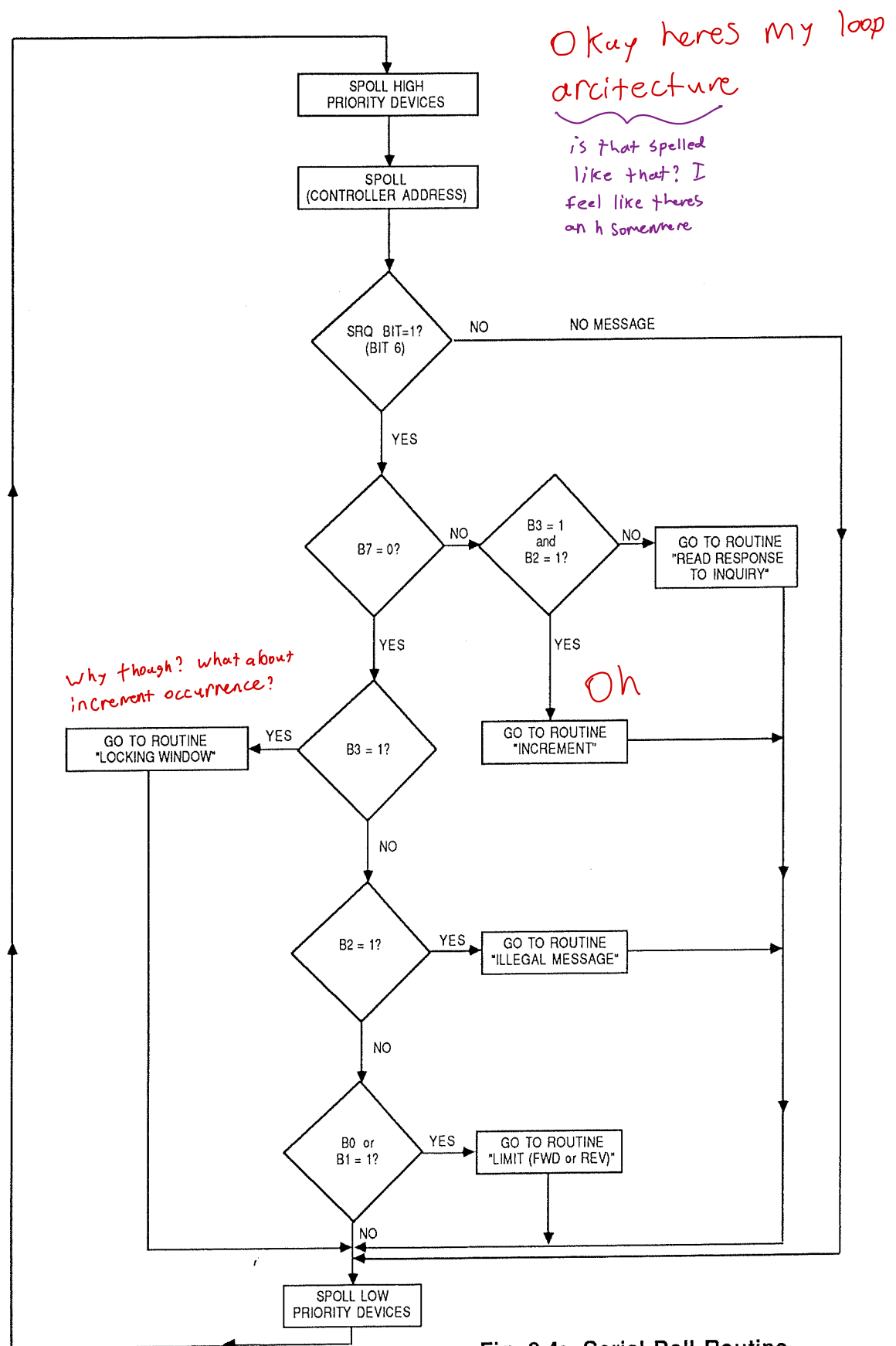
should have my checking loop look
for increments specifically

In contrast to parallel poll, the serial poll method employs a sequence of queries that are addressed serially to specific devices. User-set priorities determine the order in which the devices connected to the bus are polled. Each device responds in turn to the instruction:

SERIAL POLL (SPOLL device address)

by returning the Serial Poll Byte described in the previous section. However, with serial polling, unlike parallel polling, the computer must check the SRQ bit to determine whether or not the device has a message to send.

Note that when polling serially, we have to check bits B2 and B3 in SPOLL BYTE in order to determine whether an increment had occurred (something that the PARALLEL POLL byte indicates directly, making further checking unnecessary).





2.1.3 PPOLL or SPOLL?

should use Spoll, no other
devices attached

The answer to this question should really be divided into two:

- * if only one device is connected on the Bus (besides the computer, of course!), there is no sense in PPOLLing.
- * if more than one device is requesting service on the Bus, working with PPOLL will actually speed up the computers response to an SRQ.

The computers response will be quicker because by executing a PPOLL, it is able to immediately establish which of the devices requested service. There is not the case when using SPOLL only, here the computer has to sequentially poll all the devices on the Bus to get to the one requesting service.

It is obvious that if only one device is connected, there is no problem of "isolating" its request among others. In this case using PPOLL not only does not speed things up, but can actually slow them down! (That is because, after a PPOLL, one has to make a SPOLL addressed to the particular, requesting service device, anyway).



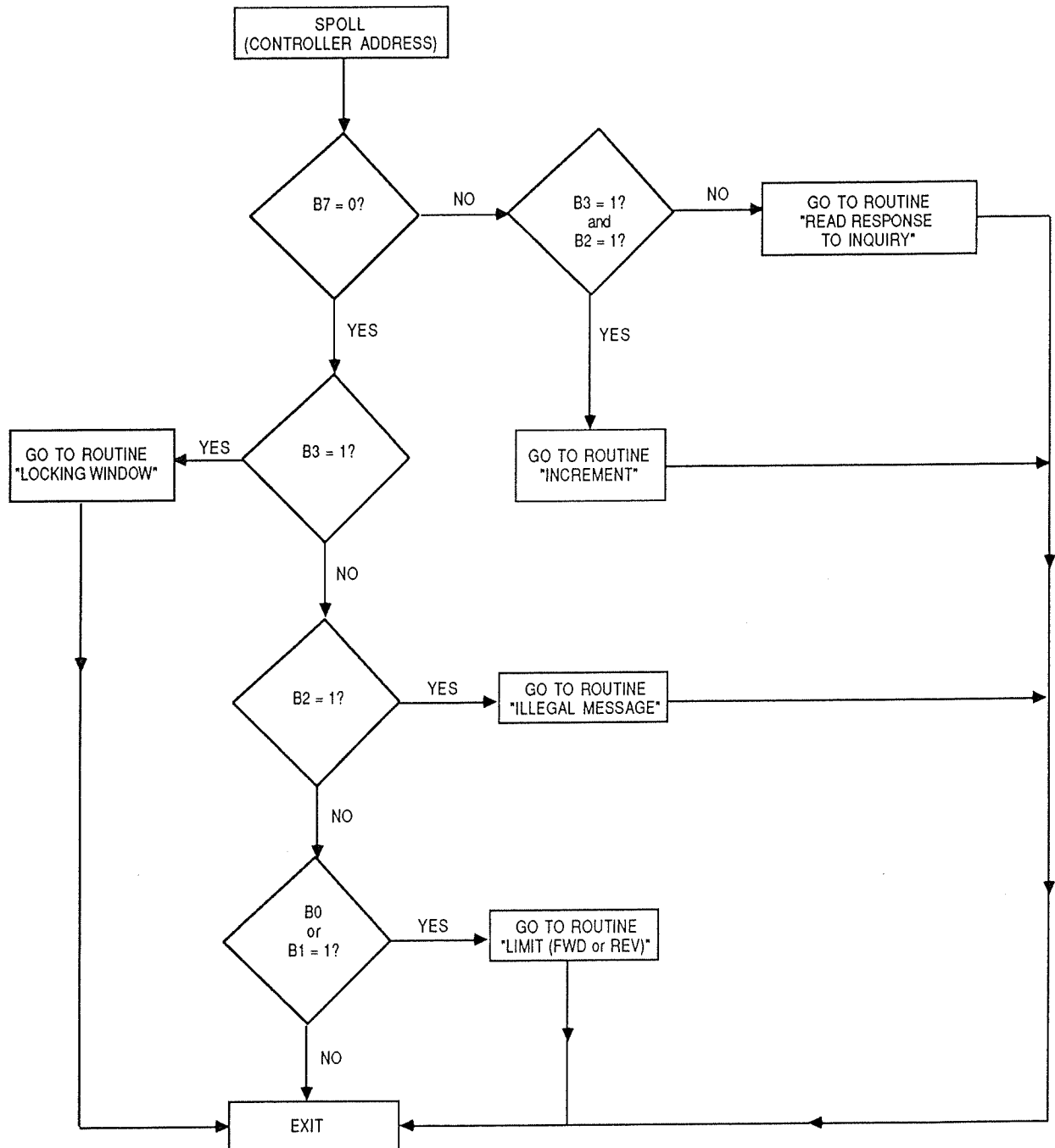
2.1.4 Service Request Interrupt

This method necessitates the incorporation of interrupt software in the computer. In addition, if software priorities are to be set, an interrupt priority scheme must be implemented. This is normally no problem with IEEE-488.

When the interrupt signal sent by a hardware line is received by the computer, the computer automatically branches to the CONTROLLER INTERRUPT service routine. This routine is similar to the one described in the previous section.

The main advantage of the interrupt method is that it allows the computer to work on a main task, being interrupted only occasionally by the controller or other devices.

The interrupt referred to would normally be obtained by sensing the SRQ line on the IEEE 488. But it may as well be the "BUFFER FULL" interrupt of RS-232-C.



NOTE: There is no need to check the SRQ BIT, because this bit triggers the interrupt!

Fig. 2.5: SRQ Interrupt Service Routine



2.1.5 Increment Values Format

Current angular positions are issued by the CONTROLLER whenever the positioner crosses an incremental value defined by a LOAD increment command (see paragraph 3.3.7). The actual value takes the format:

XXXXXX or -XXXXXX

where XXXXXX is a string of non-negative numbers, equivalent to the said actual position in degrees. Had it been written in decimal form, it would have taken the format XXX.XXX.

The "-" sign will appear when the $\pm 180.000^\circ$ format is used. (See para. 3.3.10.6).



2.2

RS-232-C

This link, naturally, can operate only one device. Because of that, all the tools for dealing with multiple device Bus (like Address or PPOLL) are not necessary. The way in which the computer inputs information from CONTROLLER is quite simple: Each incoming frame interrupts the computer. The first byte in each frame is always a byte which has the same structure as a SPOLL BYTE in IEEE-488 (Table 2-1).

Because of that, entering data from RS-232-C is exactly the same as entering data from IEEE-488 in SRQ Interrupt Method (see Paragraph 2.1.3).

Detailed notes on the use of RS-232 are not included since these are no "standard" procedures. It is pointed out that this Bus is inherently slower than IEEE-488 and this may be a limiting factor in some applications.



2.3

Controller Output Buffers

The dialogue between the controller and the computer often results in a multitude of messages that must be sent from the former to the latter. However, as mentioned before, the computer has sole discretion in determining the exact timing of the reception of the controller messages.

In order to prevent the loss of messages due to message jams, the controller queues the messages in a stacked FIFO buffer, (whether IEEE or RS-232).

If there is a message pending, the first byte waiting in the buffer will have the SRQ (Bit 6) set to 1 and will set the PPOLL bits. This byte appears on the IEEE Bus and may be used to initiate a PPOLL or SPOLL routine. It will also appear on the RS-232 serial bus and may be treated in a way similar to that used in GPIB SPOLL routine.

In the case of the IEEE bus, usually the first step in computer reply is to send a SERIAL POLL instruction addressed to the device. This action momentarily sets the SRQ line to 0, and deletes the respective SERIAL POLL BYTE from the stack in the computer. Should the buffer stack contain another SPOLL BYTE, it will now occupy the place vacated by the previous one, and the cycle repeats itself.

Important



I'm starting to think they had something more standard in mind, so I may not have to worry about some of this. Probably already handled by the library

If, while honoring the service request, the computer must read a data message residing in the data output buffer, a similar process takes place. The reading is carried out by the computer instruction: ENTER. The first ENTER instruction reads the first data byte in the output buffer. Once the ENTER instruction is executed, the respective data byte is deleted from the buffer and the next byte occupies the data lines.

If, for some reason, the computer fails to honor an SRQ, the CONTROLLER will hold any further messages until the SRQ is honored. Failure to honor a pending SRQ may eventually result in a full buffer and subsequent loss of data.

The increment message request is also honored by an ENTER instruction. Once the conditions for sending a synch increment signal are met (the positioner has crossed an angular increment position), the controller issues a synch signal through a rear panel BNC connector. The controller stores in the output buffer the exact position that the positioner had when the synch signal was sent. The first operation performed by the "Increment Service" routine, is to read this position with an ENTER instruction. (This action resets the INCREMENT bit in the PARALLEL POLL byte and deletes the corresponding SERIAL POLL and data bytes from the buffer).

The buffers mentioned above can store up to 60 bytes. If more than this number accumulate, messages may be lost.

It is up to the system designer to see that the computer time is allocated according to the expected event rate, so that all the service requests are honored and no messages are lost.



A full buffer can cause problems, for instance if an event occurs (say limit is tripped), it is stacked in the bottom of the buffer and will not be output till the whole buffer is read, or (if the buffer is already full) never be placed in the buffer and no SRQ will be activated either. It would be good practice to clear the buffer by the IEEE-488 standard:

"CLEAR DEVICE"

command whenever a condition of full buffer is suspected. This does however mean that all pending messages will be lost.

Note: After a "CLEAR DEVICE" a delay of at least one second must be made in order to allow the CONTROLLER to reset its buffers.



IMPORTANT NOTE:

The value of the increment positions may occasionally exhibit a slight deviation relative to the increments designated by the positional values user.

At slow speeds of rotation, this deviation will not manifest itself; at high speeds, however, the deviation will increase as the speed increases. The reason for this behavior is that the controller sends the synch signal based on a comparison between the actual position of the positioner, and the target position at which the controller is supposed to issue the synch signal.

The comparison routine is performed cyclically, at regular intervals determined by an internal clock. When the positioner speed is fast with respect to the clock, situations may occur resulting in the detection of the fact that the positioner had just gone beyond the increment angle. When the synch signal was generated, the position at the time of delivery is recorded.

It is important to stress that even though the synch signal is slightly delayed with respect to the target, the recorded angle still reflects the accurate position of the positioner, at the time the synch signal was sent.



CHAPTER 3

COMMUNICATION LINK COMMANDS

3.1 Scope

As stated before, the COMM LINK COMMANDS are ASCII characters interpreted by the CONTROLLER as different motion control and measurement commands. A few COMM LINK COMMANDS may be stringed together into one message (or frame). Each message is terminated by an ASCII "CR", "LF" (DEC 13, 10).

Note: ASCII characters, in the following discussions, will be in quotation marks ("CR", "LF"); or separated by ABS signs (P|a|s|0|2|4|5|8|<).



3.2 Main Commands and their Hierarchy

Table 3.1: The Main Commands:

ASCII	MNEMONICS	DESCRIPTION
"L"	LOAD	Load parameters into CONTROLLER
"S"	READ	Read parameters (previously loaded by LOAD)
"G"	RUN (*)	Start the motion (in one of the motion modes previously set by LOAD)
"K<"	SAVE	Save parameters (previously loaded by LOAD) into non-volatile memory
"FT1<"	TEST	Initiate a CONTROLLER self-test
"H"	STOP (*)	Stop all motion, turn the CONTROLLER to "STAND-BY" state

(*) Note: RUN and STOP commands are acceleration/deceleration controlled.



If the CONTROLLER is under front PANEL control, all these commands are ignored.

In "COMPUTER" mode each of the commands turns the CONTROLLER from "STAND-BY" to a certain STATE (except H, which will retain it in "STAND-BY").

When turned to one of the STATES, the CONTROLLER will not accept any other MAIN COMMAND except the "H", which returns it to "STAND-BY" state.

This is illustrated by the following diagram:

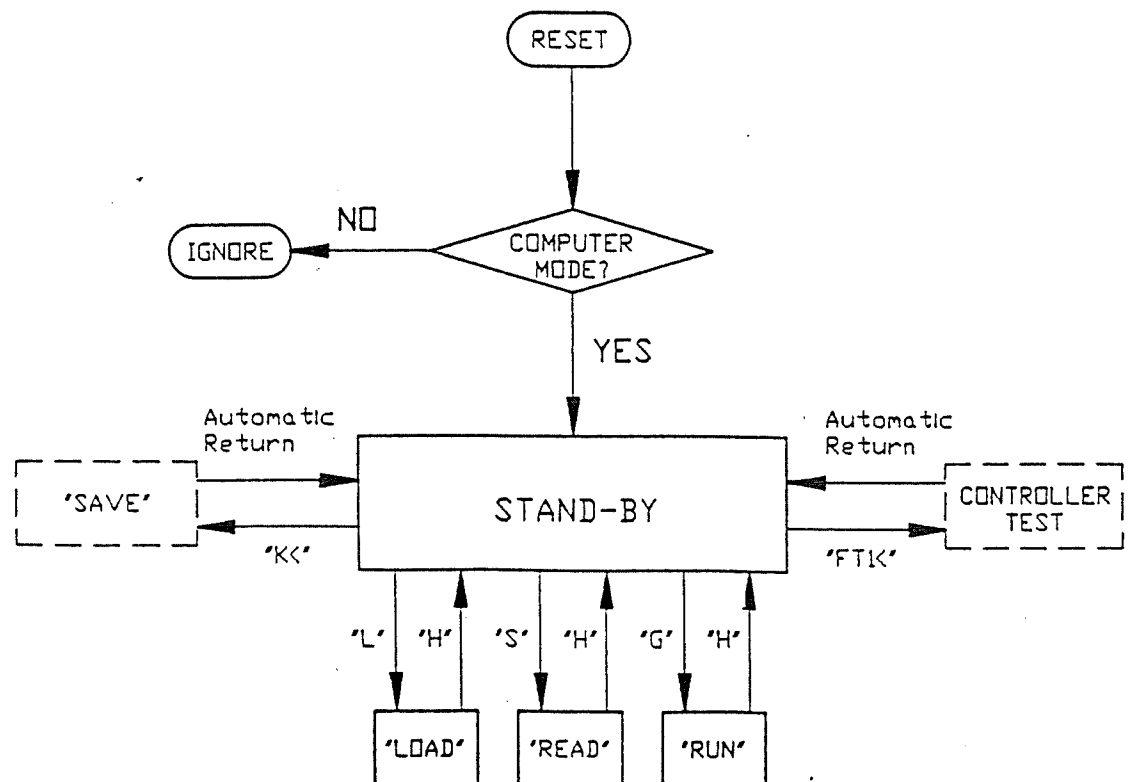


Fig. 3-1: Main Commands Hierarchy



"L" - turns the CONTROLLER to "LOAD PARAMETERS" state. In this state the CONTROLLER is ready to accept one or more parameters for motion and measurement control.

"S" - turns the CONTROLLER to "READ PARAMETERS" state. In this state the CONTROLLER is ready to accept queries for one or more parameters previously locked by LOAD. Each query is answered immediately.

"G" - turns the CONTROLLER to "RUN" state. In this state the CONTROLLER moves the positioner in one of the motion modes previously set by the LOAD. The CONTROLLER will also accept "RUN" subcommands: PAUSE, CONTINUE, SLEW RIGHT, SLEW LEFT.

Note: If the "G" command is given after an "H" (which is the situation in most cases), a short delay of 10 milliseconds should take place between the two.

"K<" - turns the CONTROLLER to "SAVE" state. This state is active only for short period of time (0.25 sec.) - during which the CONTROLLER is busy saving all the loaded parameters into non-volatile memory.

During this 0.25 sec., the CONTROLLER is therefore unable to receive any COMM LINK commands.



AL-4706-3A COMMUNICATION LINK

It is, therefore, necessary to delay any further communication with the CONTROLLER after each "K<" command for 0.25 seconds at least.

After the saving current parameters in non-volatile memory is accomplished, the CONTROLLER returns automatically to "STAND-BY".

Note: No special action is required in order to retrieve data from the non-volatile memory. All the parameters saved there are automatically loaded into CONTROLLERS workspace after a RESET.

"FT1<" - turns the CONTROLLER to "CONTROLLER self-test" state.

This state is active for a short period of time (100 msec.) during which the CONTROLLER runs the following tests:

- * CPU Card
- * F-I/O Card
- * Fast Position Cards
- * Front Panel Card (only in Controllers other than AL-4706-3A)
- * Interface between Cards.

The CONTROLLER outputs the tests results by means of an appropriate EVENT message on the GPIB (see Table 2.5).

During the self-test procedure, the CONTROLLER is unable to receive any COMM LINK commands.

At completion of the self-test, the CONTROLLER returns automatically to "STAND-BY".



AL-4706-3A COMMUNICATION LINK

Example

Usually the proper order of work with the Controller is:

- * Turn the CONTROLLER to "LOAD PARAMETERS" state and load all the needed parameters defining motion and measurement. Go back to "STAND-BY".
- * Turn the CONTROLLER to "READ PARAMETERS" state and read all the loaded parameters in order to assure that they were accepted properly. Go back to "STAND-BY".
- * Save parameters in non-volatile memory (Optional).
- * Turn the CONTROLLER to "RUN" state and execute the desired motion.



Flow chart
for the above
example

This sequence is illustrated by the following flowchart:

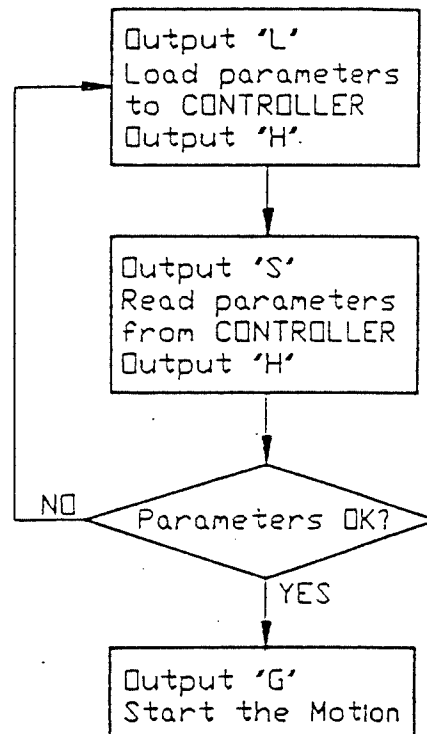


Fig. 3.2: "Proper Order of Work" (without monitoring)

Note: This flowchart is somewhat uncomplete - it will be modified in Paragraph 3.6, in which the "Current Status Queries" are described.



3.3 "LOAD Parameters"

After turning the CONTROLLER into "LOAD PARAMETERS" state by outputting an ASCII "L", the following parameters may be loaded:

3.3.1 "P" - Boundary Position

Boundary position consists of three letters as a header:

"Pas", "Pab", "Pes", "Peb", "Pat", "Pet", and five angular position digits followed by a "<" terminator:

* P|a|s|0|1|8|3|9|< - starting position of scan, in RASTER mode, is set to 18.39°

* P|a|b|3|4|8|5|4|< - ending position of scan, in RASTER mode, is set to 348.54°.

* P|e|s|0|0|0|4|2|< - starting position of step, in RASTER mode, is set to 0.42°.

* P|e|b|1|0|2|0|0|< - ending position of step, in RASTER mode, is set to 102.0°.



AL-4706-3A COMMUNICATION LINK

Note: * The span between step starting point and step ending point is limited to 180° .
If a span of more than 180° is defined, the controller will execute a RASTER with one step to an unpredictable position.

* P|a|t|2|7|9|8|4|< - target position of first axis, in track mode is set to 279.84° .

* P|e|t|0|4|2|0|1|< - target position of second axis, in track mode, is set to 42.01° .

Range of permissible values: $0^\circ - 539.99^\circ$

Error conditions: * Position exceeds 539.99°

Example: P|a|t|6|0|1|8|9|<

* Number of position digits differs from 5:

Example: P|e|s|1|2|<

* Undefined character after "P".

Note: When loading parameters, all leading and trailing zeros must be included.
i.e. P|a|s|1|2|< for Raster Start angle = 1.2° will not be accepted.
It must be P|a|s|0|0|1|2|0|<.
This remark applies to all loading parameters.



3.3.2 "A" Axis Selection

3.3.2.1 For one-axis-at-a-time Controller:

Okay what the
hell is Scan,
Step, and Slew
in this context???

Axis Selection consists of two letter header - "Aa", "Ae", "Az" (except for offset updating axis selection, in which there is only one letter "A") and one digit, designating the selected axis, followed by a "<" terminator.

- * A|a|1|< - In RASTER mode:
scan axis is set to be 1.
In TRACK mode:
axis of first rotation is 1.
- * A|e|5|< - In RASTER mode:
step axis is set to be 5
In TRACK mode:
axis of second rotation is 5
- * A|z|4|< - Slew axis is set to be 4



- * $A|2|<$ - Configuration parameters updating axis is 2.
This means that configuration parameters (see para. 3.3.10) will refer to axis 2.

What is raster and track mode?

Note: Zero selection in $A|e|n|<$ command is allowed.

$A|e|0|<$ - In raster mode: step axis not selected.

In track mode: second rotation axis not selected

Not selecting the axis "e" (" $AeO|<$ " command) can be quite useful. For example in defining a sector scan motion (see Para. 3.3.5) or to move only one axis to a target angle in track mode.

- 2) Note that the actual axis switching (actual relay activation for selection of one out of six axes) takes place only with motion activation command - "G": the axis selection relays in Controller & Amplifier are switched according to predefined by $A|a|n|<$, $A|e|n|<$ or $A|z|n|<$ commands values, and the positioner begins moving.

Range of permissible axis values: 0 to 6
(or limited by the number of axes used in the current configuration.)

Error conditions: * Axis number out of range (higher than 6)

* Undefined character after "A"



3.3.2.2 For two-axes-at-a-time Controllers (Option 2):

Axis Selection consists of two letter header - "Aa", "Ae" (except for configuration updating axis selection, in which there is only one letter "A") and one digit, designating the selected axis, followed by a "<" terminator.

need to define
Raster, track, and
Slew mode,

alongSide Scan,
Step, and Slew
in the angle
Settings

* A|a|1|<

-

In RASTER mode:

scan axis is set to be 1.

In TRACK or SLEW mode:

axis A is 1.

Two
Options
listed
here

* A|e|5|<

-

In RASTER mode:

step axis is set to be 5

In TRACK or SLEW mode:

axis B is 5



* $A|2|<$ - Configuration parameters updating axis is 2.

This means that configuration parameters (see para. 3.3.10) will refer to axis 2.

Note: Zero selection in $A|e|n|<$ command is allowed.

$A|e|0|<$ - In raster mode: step axis not selected.

In track mode: second rotation axis not selected

Not selecting the axis "e" (" $Ae0|<$ " command) can be quite useful. For example in defining a sector scan motion (see Para. 3.3.5) or to move only one axis to a target angle in track mode.

- 2) Note that the actual axis switching (actual relay activation for selection of one out of six axes) takes place only with motion activation command - "G": the axis selection relays in Controller & Amplifier are switched according to predefined by $A|a|n|<$, $A|e|n|<$ commands values, and the positioner begins moving.

Range of permissible axis values: 0 to 6
(or limited by
the number of
axes used in the
current
configuration.)

Error conditions: * Axis number out of range
(higher than 6)

* Undefined character after "A"

3.3.3 "V" - Velocity

Just Set
the percentage
of the attached
motors speed

3.3.3.1 For one-axis-at-a-time Controllers:

* The velocity parameter consists of two letter and two zero headers - "Va00", "Ve00", "Vz00" and 3 digits, representing the percentage of max. positioner velocity, followed by a "<" terminator:

* V|a|0|0|2|7|8|< - Velocity of scan in RASTER mode or velocity of first axis in TRACK mode is set to be 27.8% of max.

Raster mode Speeds

- Scan
- Step and speed towards Start
- Slew

* V|e|0|0|9|9|9|< - Velocity of step, velocity of moving towards the starting point in RASTER mode, velocity of scan return in RASTER A or second axis travel velocity in TRACK mode is set to max.

must have the "00"
headers *

* V|z|0|0|0|7|5|< - Velocity of SLEW is set to be 7.5%



Range of permissible velocity percentage values:
0 - 99.9%

Error conditions:

* Overall number of digits differs from 5,

example: V|e|2|8|9

* Undefined character after "V"

Note: The velocity programmed serves as a reference to the velocity closed loop. This is different from the velocity obtained when the controller responds to a current status command ("X3<"), which is actual angular velocity obtained from the position sensor.

Simplified:

enter data in percentage,

Vel readings in actual angular vel units



3.3.3.2 For two-axes-at-a-time Controllers:

okay what are
"two axes at a
time controllers"?

The velocity parameter consists of two letter and two zero headers - "Va00", "Ve00", and 3 digits, representing the percentage of max. positioner velocity, followed by a "<" terminator:

- * `V|a|0|0|2|7|8|<` - Velocity of scan in RASTER mode or velocity of first axis in TRACK or SLEW modes is set to be 27.8% of max.
- * `V|e|0|0|9|9|9|<` - Velocity of step, velocity of moving towards the starting point in RASTER mode, velocity of scan return in RASTER A or second axis travel velocity in TRACK or SLEW modes is set to max.



AL-4706-3A COMMUNICATION LINK

Range of permissible velocity percentage values:
0 - 99.9%

Error conditions:

- * Overall number of digits differs from 5,

example: V|e|2|8|9

- * Undefined character after "V"

Note: The velocity programmed serves as a reference to the velocity closed loop. This is different from the velocity obtained when the controller responds to a current status command ("X3<"), which is actual angular velocity obtained from the position sensor.

3.3.4 "D" - Direction

Set direction of
movements. The
angles only give
reference points,
not how to get
there

The Direction parameter consists of three letters followed by a "<" terminator:

Direction selection has different meanings when used in "TRACK" or "RASTER" modes.

In "TRACK":

- * D|a|f|< - Direction of first rotation is set to be "forward".
- * D|a|r|< - Direction of first rotation is set to be "reverse".
- * D|a|d|< - Direction of first rotation is set by the Controller according to the shortest way to target. (**)
- * D|e|f|< - Direction of second rotation is set to be "forward".
- * D|e|r|< - Direction of second rotation is set to be "reverse".
- * D|e|d|< - Direction of second rotation is set by the Controller according to the shortest way to target. (**)



"A or B Sector" ?

AL-4706-3A COMMUNICATION LINK

In "RASTER" (A or B or SECTOR):

- * $D|a|f| <$ - The RASTER is executed (including record increments generation) from starting point to ending point in CW direction.
- * $D|a|r| <$ - The RASTER is executed (including record increments generation), from starting point to ending point in CCW direction.
- * $D|a|d| <$ - The RASTER is executed (including record increments generation) from starting point to ending point in CW direction.
- * $D|e|f| <$
 $D|e|r| <$
 $D|e|d| <$ - not applicable

- The movement to the starting point will be always in the shortest possible way.

- See detailed description in Operation Manual - Para. 5.4.11.

Error conditions: * Undefined character after "D".

(**) Note: One must take into consideration that while moving towards the target ("TRACK" mode) or towards the starting point ("RASTER mode) in the shortest way a limit switch may be encountered. This can be prevented by defining software limits (see para. 3.3.10.2). If software limits are set, the shortest way direction will become shortest possible way direction according to the software limits setting.

3.3.5 "M" - Mode of Motion(*)

Mode of motion parameter consists of two letters.
(No terminator is needed!)

- * M|R - Raster Scan A or Sector Scan A
- * M|N - Raster Scan B or Sector Scan B
- * M|T - Track to point
- * M|Z - Slew
- * M|U - Direct (see description on next page)

Note: Unlike in front panel operation, there is no special command for Sector scan.

The Sector scan is set by using one of the "Raster" modes (A or B) and selecting a dummy (0) as the stepping axis (A|e|0|<).

The controller recognizes this situation as Sector scan. In this case it will also interpret the step value (see Para. 3.3.6) as a number of scans to be made.

As the "origin" of sector scan is Raster, it is easy to see that the difference between Sector A and Sector B is that in Sector A increments may be obtained only on the CW sweep, while in Sector B, increments may be outputted in both CW and CCW scans. It may also be concluded that a sector movement may be done only for one axis at a time.

Error condition: * Undefined character after "M".

(*) A detailed description about all the modes except "Direct" is included in OPERATION MANUAL.



00000 this
is neat

Direct Mode:

This is a special mode designed for "tracking a moving target" or similar applications. It is also quite useful for stepping the positioner in small, accurate, fast steps (see para. 4.6.4).

When in Direct, the Controller enters the "RUN" state and is ready to accept sequential target updating via the GPIB by means of a LOAD command. After receiving a target and a "GO" command, the Controller will move the Positioner towards the target. Meanwhile, if a new target update is received the Controller will move towards the new target even if the previous has not been reached yet. In this way various travel trajectories may be created.

While working with the Direct mode a few points need special consideration:

- * The speed in which the Positioner will move is defined by the rate and value of the target updates and not by the loaded speed parameter (as described on page 36). Main speed between two update points $\langle V_K \rangle$ is given by:

$$\langle V_K \rangle = \frac{\Delta \text{Target}}{\Delta \text{Time}} \quad \text{oh that's clever}$$

The loaded speed parameter will serve only as a higher bound for the dynamic speed $\langle V_K \rangle$: if target update steps are too large or they are coming in too high a rate, so that $\langle V_K \rangle$ exceeds its bound, $\langle V_K \rangle$ will be limited by the bound and as a result the system will not be able to track after the target.

- * No "LOCK" messages are outputted by the CONTROLLER in Direct mode.
- * In Direct mode, Positioner is moved in fastest possible way - no acceleration/deceleration ramps are incorporated: because of that, if it is only needed to go from one point to another - TRACK mode should be used.
- * Direct mode movement is always the shortest way towards the target, so even if an overshoot occurs, the system is able to correct itself in real time.
- * Worst case track (dynamic) error depends on system parameters:
type of positioner, positioner load, type of trajectory (max. speed and acceleration derived from it).



- * Direct mode is calibrated for best performance by the same parameters as any other mode of motion: ACCL/DECL and GAIN factors (see paras. 3.3.10.4 & 3.3.10.5). But, here, the ACCL/DECL parameter serves quite differently: it is referred to by the CONTROLLER as a FEED_FORWARD parameter which dynamically re-enforces the GAIN when a velocity change of the reference target is detected. In general, if the mode is used for tracking a moving target, the ORBIT Passing Target Software Simulator is recommended for calibration purposes.



3.3.6 "J" - Step Value (in Raster Mode) or Number of Scans (in Sector Mode)

The "J" parameter represents the step value or number of scans according to the selected mode: Raster or Sector Scan (see Para. 3.3.5). In either case it consists of one letter ("J") and one zero header and 4 digits which are the angular value of step or number of scans accordingly. The digits are followed by "<" terminator.

J|0|0|2|4|0|< - Step value in Raster Mode is

2.40°. Number of scans in Sector mode is 240.

Range of permissible values of step value:
0 - 99.99°

Range of permissible values of scans:
0 - 255

Error conditions:

- * Numeral data exceeding 9999.
Ex.: J|1|2|8|4|0|<
- * Overall number of digits differs from 5,
Ex.: J|8|9|4|<
- * Undefined character after "J".

Note: If more than 255 scans are loaded, no error message will be outputted, but the CONTROLLER will execute the number of scans given MOD (255).



3.3.7 "I" - Increment Step in RASTER MODE (*)

Increment step parameter consists of one letter header ("I") and 5 digits defining the angular gap between two consecutive increments, followed by a "<" terminator:

I|0|5|8|4|1|< - Increment step is set to 5.841°

Note: The resolution is one thousands of a degree

Loading I|0|0|0|0|0|< switches off the increments

Range of permissible values:

Min ÷ 49.999°

Where Min is:

For GPIB increments - position programmed
velocity (in deg./sec.)
divided by 100Hz.
Ex.: If positioner velocity
is programmed to
10°/sec., then

$$\text{Min} = \frac{10^\circ/\text{sec}}{100\text{Hz}} = 0.1^\circ$$

For TTL increments (with GPIB increments disabled by
"Fz<" (see para. 3.3.8) - position velocity (in
deg./sec.) divided by
2000Hz.

Ex.: If positioner velocity
is programmed to
10°/sec., then

$$\text{Min} = \frac{10^\circ/\text{sec}}{2000\text{Hz}} = 0.005^\circ$$

(In any case, the smallest possible increment is
0.001°).

(*) More detailed information about the increments may be found
in the OPERATION MANUAL.

fuck youuuuuuuuu



Error conditions:

* Increment step exceeding 49.999°, ex.:

I|5|9|0|4|1|<

* Undefined character after "I".

Note: No error message will be output by the controller if the selected increment step is less than Min, but the increments will be in error as described in note in Para. 2.3.



3.3.8 "F" - Bus Increments Enable/Disable

Increments Enable/Disable parameter consists of two letters, followed by an "<" terminator: -

F|e|< - Disable Increment position output

This leaves just the PPOLL and SPOLL bytes active for each increment (see chapter 2). But, contrary to the normal situation, the CONTROLLER will not output angular updates (as described in para. 2.1.5), so that no ENTER action is to be taken by the computer. (Increment pulses which are output on the BNC connector are not affected).

Note: This command will be accepted by CONTROLLER with firmware version of 1.6.1 or later.

F|z|< - Disable Increments via the Communication

Bus (Increments pulses which are output on the BNC connector are not affected!).

F|b|< - Enable Increments via the Communication Bus.

Note: In any case one must take into consideration that loading I|0|0|0|0|0|< switches off all increments (increment pulses on BNC connector as well as increment angles on the Communication Bus).

Error conditions:

* Undefined characters after "F".



As explained in Para. 2.2.1, the PPOLL BYTE configuration is made not by the IEEE-488 PPC command, but by a "COMM. LINK" LOAD command.

```
C|5|1|<      -   Message   bit is set to be No. 5
                  Increment bit is set to be No. 1
```

OTHER DEVICES		MESSAGE	OTHER DEVICES			INCREMENT	
BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0

```
* Digit higher than 7
* Number of digits differ from 2
* Undefined character after "C".
```



3.3.10 Configuration Parameters

Configuration parameters are:

- Offset
- Software limits
- Lock window
- Position measurement type
- Control loop gain
- Open loop acceleration/deceleration

Each axis has its individual set of these parameters. To enable updating each axis parameters, each configuration parameter load command may be pre-fixed by an "An<" command (see para. 3.3.2).

This axis selection prefix does not have to proceed each command, it may be output only once after which it is latched by the CONTROLLER so that, until a new axis selection prefix is output, all parameters will be loaded relating to the previous selected axis.

Ex.: H|L|A|2|<|W|0|0|0|1|5|<|g|0|0|0|1|7|<|A|5|<|B|r|
1|5|9|8|8|<|B|f|0|9|0|0|0|<

which means:

Load axis 2 lock window = 0.15°
axis 2 acceleration/deceleration = 17
axis 5 reverse software limit = 159.88°
axis 5 forward software limit = 90.00°

For more detailed information about the meaning and effect of each of these parameters on the system, see the Operation Manual.

*Can stack
commands*



3.3.10.1 "O" - Offset

Offset parameter consists of one letter header ("O"), and 5 angular position digits representing the offset to previously selected axis. The digits are followed by a "<" terminator:

O|2|0|5|1|0|< - the offset is 205.1°

Range of permissible values: 0 - 359.99°

Error conditions: * Offset exceeding 359.99°,

ex.: O|5|7|3|8|1|<

* Number of digits differs

from 5, example: O|2|4|8|<

* Undefined character after "O".



3.3.10.2 "B" Software Limits

The software limits parameter is used by the Controller when calculating the direction of movement towards a target in shortest path without encountering the limits (shortest possible way).

The software limit parameter consists of two letters and five digits representing the angular software limit to the previously selected axis. The letter following "B" represents either forward limit ("f") or reverse limit ("r").

The digits are followed by a "<" terminator:

- * B|f|1|7|4|5|1|< - Forward software limit
 is set to be 174.51°
- * B|r|0|3|5|0|1|< - Reverse software limit
 is set to be 35.01°.

Loading a forward limit position which is equal to a reverse limit position, switches off the software limits.

Range of permissible values: 0 - 359.99°.

The enabled by software limits movement sector is limited to 350°.

Error conditions:

- * Angular value exceeds 359.99°
Example: B|f|6|0|3|4|4|<
- * Number of angular digits differs from 5
Example: B|r|3|4|5|<
- * Undefined character after "B".



3.3.10.3 "W" - Lock Window

Lock window parameter consists of one letter ("W") and 3 zero header and 2 digits defining the lock window, followed by a "<" terminator.

W|0|0|0|1|2|< - Lock window is set to be: 0.12°

Range of permissible values of the lock window:
0 - 0.99°

Error conditions:

* Lock window exceeding 0.99°, ex.: W|0|0|5|8|4|<

* Overall number of digits differs from 5,
example: W|0|1|2|<

* Undefined character after "W".



3.3.10.4 "c" - Gain Factor

Gain parameter consists of one letter "c" and 2 zero header and 3 digits representing the gain to previously selected axis. The digits are followed by a "<" terminator:

c|0|0|0|0|2|< - the gain is 2.

Range of permissible values: 1 - 255

Error conditions:

- * gain exceeding 255
- * Number of digits differ from 5, ex.: c|5|3|4|<
- * undefined character after "c"

WARNING:

This parameter affects the closed loop response. It is calibrated per system and should not be changed by unauthorized personnel.



3.3.10.5 "g" - ACC/DEC (Acceleration/Deceleration) Factor

Acceleration/Deceleration factor consists of one letter ("g") and 2 zero header and three digits representing the ACC/DEC factor to previously selected axis. The digits are followed by a "<" terminator:

g|0|0|0|2|3|< - the ACC/DEC factor is 23

Range of permissible values: 1 - 255

Error conditions:

- * ACC/DEC factor exceeding 255

ex.: g|0|1|4|3|8|<

- * Number of digits differ from 5,

ex.: g|3|6|5|<

- * undefined character after "g"

WARNING:

- This parameter affects the closed loop response. It is calibrated per system and should not be changed by unauthorized personnel.



3.3.10.6 "n" - type

Type parameter consists of one letter ("n") header and one digit representing the type of positioner measurement device and the format of the position readouts which were output by the CONTROLLER on its various outputs. This digit is followed by a "<" terminator:

n|1|< - the type of the previously selected axis is 1
(Dual-speed synchro, 0-360° format readout).

Range of permissible values:

- 0 - One-speed synchro, 0-360° format readout
- 1 - Dual-speed synchro, 0-360° format readout
- 2 - Inductosyn^(R), 0-360° format readout (Option 3)
- 3 - Optical encoder, 0-360° format readout (Option 4)
- 4 - Inclinator, 0-360° format readout (Option 4)
- 5 - One-speed synchro, ±180° format readout
- 6 - Dual-speed synchro, ±180° format readout
- 7 - Inductosyn^(R), ±180° format readout (Option 3)
- 8 - Optical encoder, ±180° format readout (Option 4)
- 9 - Inclinator, ±180° format readout (Option 4)

Note that for actually activating the new selected type on a particular axis, it is not enough to update its configuration by the "n" command. The change of type is activated only with a transient of axis selection in the PCU or by a CONTROLLER hardware reset. For a more detailed explanation - see the Operation Manual.

Error conditions:

* type exceeding 9
ex.: n|1|2|<

* undefined character after "n".



3.3.11 "i" - Baud-Rate Parameter for RS-232-C Communication (Option 6)

The "i" parameter represents the serial communication Baud-rate and it can be set as 19200, 9600, 4800, 1200, ..., or 300 b/sec.

The Baud-rate parameter consists of two letters, followed by a "<" terminator:

- i|0|< - Set the Baud-rate to 19200 bits/sec'.
- i|1|< - Set the Baud-rate to 9600 bits/sec'.
- i|2|< - Set the Baud-rate to 4800 bits/sec'.
- i|3|< - Set the Baud-rate to 2400 bits/sec'.
- i|4|< - Set the Baud-rate to 1200 bits/sec'.
- i|5|< - Set the Baud-rate to 600 bits/sec'.
- i|6|< - Set the Baud-rate to 300 bits/sec'.

Important Note

After the Baud-rate parameter is set to the new value, it should be saved ("K<") in non-volatile memory and the Controller should be turned OFF and ON in order to continue the communication with an updated Baud-rate.

Error Conditions

- * Parameter "i" exceeding 9
ex.: i|2|1|<
- * Undefined character after "i".



3.3.12 "j" - DC-OUT Range Parameter (Option 6)

The "j" parameter is a scaling factor for the Analog DC voltage position indicator.

Factory setting is $j=1$. In this case:

if unipolar mode is selected ($n < 5$, see 3.3.10.6)
0 to 360° will correspond to 0 to 10V

if bipolar mode is selected ($n \geq 5$, see 3.3.10.6)
-180 to $+180^\circ$ will correspond to -10V to 10V

For "j" = 1 the DC voltage indicator will change its scale in the following manner:

if unipolar mode is selected:

360°
0 to $\frac{360^\circ}{j}$ will correspond to 0 to 10V

if bipolar mode is selected:

180° 180°
- $\frac{180^\circ}{j}$ to + $\frac{180^\circ}{j}$ will correspond to -10V to +10V

ex.: $j = 12$

for unipolar mode 0° to 30° will cause 0 to 10V
 30° to 60° will cause 0 to 10V
 60° to 90° will cause 0 to 10V
 and so on ...

for bipolar mode -15° to 15° will cause -10V to +10V
 15° to 45° will cause -10V to +10V
 45° to 75° will cause -10V to 10V
 and so on ...

The DC-OUT range parameter consists of one letter "j", two zero headers and three digits defining the scaling factor (as explained above) followed by a "<" terminator:

j|0|0|1|8|0|< - the scaling factor is 180.

Range of permissible values:

$0 \leq j \leq 255$

$j=0$ & $j=255$ are referred as $j=1$)

Error Conditions

- * "j" parameter exceeding 255.
- * Undefined character after "j"



3.4 "READ" Parameters

In order to read previously loaded parameters the CONTROLLER has to be turned into "READ PARAMETERS" state. This is done by outputting an ASCII "S".

Then different parameters may be queried. The CONTROLLER's responses to the queries can be entered by the computer in different ways - all of them described in Chapter 2.

The following pages describe all the possible parameter queries and the format of the responses.

Error conditions for all these queries:
Undefined character after the initial "P", "A",
etc.

Response to all queries when in READ state always take the form of:

S|n|n|n|n|n

Number of characters after "S" may vary from 5 to 3. Some of the leading zeros may be suppressed when values (angles, velocities, etc.) are unvalued. But still, leading zeros are retained to keep the response at least 3 characters long.



Note: The CONTROLLER's delay between receiving a query and answering it (SRQ going down) is 1 millisecond at most.



3.4.1 "p" - Read Predefined Boundary Position:

- * P|a|s|< - Read starting position of raster scan
- * P|a|b|< - Read ending position of raster scan
- * P|e|s|< - Read starting position of raster step
- * P|e|b|< - Read ending position of raster step
- * P|a|t|< - Read track target of first rotation axis
- * P|e|t|< - Read track target of second rotation axis



CONTROLLER Response Format:

One letter header ("S") and angular position digits. The number of digits may differ: 5, 4 or 3 digits - because leading zeros are omitted.

Examples:

If the defined target position of the second axis in track motion is 219.15° , the response on the following query:

P|e|t|<

is:

S|2|1|9|1|5

If the target angle was 14.3° the response would be:

S|1|4|3|0

If the target angle was 0.3° the response would be:

S|0|3|0

Note: Leading zeros included. Give minimum of 3 digits.



3.4.2 "A" - Read Axis Selection

3.4.2.1 For one-axis-at-a-time

* $A|a|<$ - In RASTER mode: read scan axis
selection

In TRACK mode:
read first axis selection

* $A|e|<$ - In RASTER mode: read step axis
selection

In TRACK mode:
read second axis selection

* $A|z|<$ - Read SLEW axis selection.

CONTROLLER response format:

One letter header ("S"), two zeros, and one digit
designating the selected axis.

Examples:

If the defined scan axis was 5, the response to the
following query:

$A|a|<$

is:

$S|0|0|5$



If the response to the query:

A|e|<

is:

S|0|0|0

that means:

in RASTER mode
in TRACK mode

- the motion is sector scan
- the motion is only in first axis





```
*      A|a|<      - In RASTER mode: read scan axis
                        selection
```

* $A|e| <$ - In RASTER mode: read step axis
 selection

CONTROLLER response format:

Examples:

$$A \mid a \mid <$$

S | 0 | 0 | 5



If the response to the query:

$A|e|<$

is:

$S|0|0|0$

that means:

in RASTER mode	- the motion is sector scan
in TRACK or SLEW mode	- the motion is only in first axis



3.4.3 "V" - Read Predefined Velocity

3.4.3.1 For one-axis-at-a-time Controllers

- * $V|a|<$ - read defined velocity of scan motion of RASTER, or first rotation axis of TRACK.
- * $V|e|<$ - read defined velocity of step motion of RASTER, velocity of moving towards the starting point in RASTER, velocity of scan-return in RASTER A, second rotation of TRACK or moving towards the RASTER starting point motion.
- * $V|z|<$ - Read defined velocity of SLEW motion.

CONTROLLER response format:

One letter ("S") header and 3 velocity percentage digits.

Examples:

If for a raster mode the defined velocity of scan is 11.5%, the response to the query:

$V|a|<$

is:

S|1|1|5

If the velocity was 75%, the response would be:

S|7|5|0

If the velocity was 5%, the response would be:

S|0|5|0 (leading zeros are included).



3.4.3.2 For two-axes-at-a-time Controllers

- * $V|a|<$ - read defined velocity of scan motion of RASTER, Axis A of TRACK & SLEW.
- * $V|e|<$ - read defined velocity of step motion of RASTER, velocity of moving towards the starting point in RASTER, velocity of scan-return in RASTER A, Axis B of TRACK & SLEW, or moving towards the RASTER starting point motion.

CONTROLLER response format:

One letter ("S") header and 3 velocity percentage digits.

Examples:

If for a raster mode the defined velocity of scan is 11.5%, the response to the query:

$V|a|<$

is:

S|1|1|5

If the velocity was 75%, the response would be:

S|7|5|0

If the velocity was 5%, the response would be:

S|0|5|0 (leading zeros are included).



3.4.4 "D" - Read Predefined Direction

Note: As noted in Para. 3.3.4, the direction selection refers only for the TRACK mode.

* D|a|< - read defined direction of first axis rotation, or RASTER direction.

* D|e|< - read defined direction of second axis rotation

CONTROLLER response format:

One letter header ("S"), two zeros, and one ASCII sign designating the defined direction:

"<" - for forward

"=" - for reverse

"d" - for auto

Examples:

If the defined direction of the second axis is forward, the response to the query:

D|e|<

is:

S|0|0|<

If the direction was reverse, the response would be:

S|0|0|=



3.4.5 "J" - Read Step Value (in Raster mode), or
Number of Scans (in Sector mode)

J|< - Read step value or number of scans

CONTROLLER response format:

One letter header ("S") and 4 or 3 digits representing the step value in degrees or number of scans.

The number of digits vary because some of the leading zeros are omitted (the ones retained - keep the response at least 3 digits long).

Examples:

If the defined step value is 51.32° (or number of scans is MOD255 (5132) = 32) the response to the query:

J|<

is:

S|5|1|3|2

If the defined step value was 1.91° (or number of scans was 191), the response would be:

S|1|9|1

If the defined step value was 0.78° (or number of scans was 78), the response would be:

S|0|7|8



3.4.6 "I" - Read Predefined Increment Step

I|< - read defined increment step

CONTROLLERS response format:

One letter header ("S") and 5, 4 or 3 angular position digits (in one thousands of a degree resolution!) representing the defined gap between two consecutive increments.

The number of digits vary because some of the leading zeros are omitted (the ones retained - keep the response at least 3 digits long).

Examples:

If the defined increment step is 1.5° the response to the query:

I|<

is:

S|1|5|0|0

If the step was 0.08° , the response would be:

S|0|8|0

finally, if the received response was:

S|0|0|0

it means that the raster (or sector) motion, was defined, but no increment signals are to be delivered.



3.4.7 Read Configuration Parameters

Configuration parameters are:

- Offset
- Software limits
- Lock window
- Positioner measurement type
- Control loop gain
- Open loop acceleration/deceleration

Each axis has its individual set of these parameters. To enable reading each axis parameters, each configuration parameter read command may be pre-fixed by an "An<" command (see para. 3.3.2).

An axis selection prefix does not have to proceed each command, it may be output once after which it is latched by the CONTROLLER so that, until a new axis selection prefix is output, all parameters will be read relating the previously selected axis.



3.4.7.1 "O" - read Predefined offset

O|< - read defined offset of the preselected axis

CONTROLLER'S response format:

one letter header ("S") and angular position digits. Number of digits may differ: 5, 4 or 3 - that is because up to 2 leading zeros are omitted.

Examples:

If the defined offset of axis 2 was 348.71°, the response to the query:

A|2|<|O|<

is:

S|3|4|8|7|1



3.4.7.2 "B" - Read Predefined Software Limits

* B|f|< - read predefined forward software limit
of the preselected axis.

* B|r|< - read predefined reverse software limit
of the preselected axis.

CONTROLLER's response format:

One letter header ("S") and angular digits. The number of digits may differ: 5, 4 or 3 - since up to two leading zeros are omitted.

Examples:

If the defined forward software limit of axis 4 is 310.45°, the response to the query -

A|4|<|B|f|<

is -

S|3|1|0|4|5



3.4.7.3 "W" - Read Predefined Lock Window

W|< - read defined lock window

CONTROLLERS response format:

one letter header ("S"), a zero, and two angular position digits.

Examples:

If the defined lock window of axis 3 is 0.54°, the response to the query:

A|3|<|W|<

is:

S|0|5|4

If the defined window was 0.04°, the response would be:

S|0|0|4



3.4.7.4 "c" - read Predefined Gain

c|< - read predefined gain of the preselected axis.

CONTROLLER's response format:

one letter ("S") and three digits representing the gain.

Example:

If the defined gain of axis 5 was 2, the response to the query:

A|5|<|c|<

is:

S|0|0|2



3.4.7.5 "g" - Read Predefined ACC/DEC Factor

$g|<$ - read defined ACC/DEC factor of the preselected axis

CONTROLLER's response format:

one letter header ("S") and 3 digits representing the ACC/DEC factor.

Examples:

If the defined ACC/DEC factor of axis 2 was 135, the response to the query:

$A|2|<|g|<$

is:

$S|1|3|5$



3.4.7.6 "n" - read Predefined Type

n|< - read predefined type of the preselected axis

CONTROLLER's response format:

one letter header ("S"), two zero header and one digit representing the type.

Examples:

If the defined type of axis 4 was 1, the response to the query:

A|4|<|n|<

is:

S|0|0|1



3.4.8 "i" - Read Predefined Baud-Rate Parameter for RS-232-C Communication

i|< - Read the Baud-rate parameter.

CONTROLLER response format:

One letter header ("S"), two zeros, and one digit designating the current Baud-rate parameter.

Examples:

- If the Baud-rate is set to 9600 bits/sec, the response to the following query:

i|<

is

S|0|0|1

- If the response to the query:

i|<

is

S|0|0|3

then the Baud-rate is set to 2400 bits/sec.



3.4.9 "j" - Read Predefined DC-OUT Range Parameter

j|< - Read the DC-OUT range parameter.

CONTROLLER response format:

One letter header ("S"), a zero, and two digits designating the current DC-OUT range parameter.

Examples:

- If the range is set to 4, the response to the following query:

j|<

is

S|0|0|4

- If the response to the query:

j|<

is

S|0|3|6

then the range parameter is 36 - i.e. the angular range is 0° - 10° or $\pm 5^{\circ}$ (for "n" ≥ 5).



3.5 "RUN" state commands

As explained before, in order to execute a desired motion, one must turn the CONTROLLER to the "RUN" state.

This is done by outputting the "G" command.

"G" - turns the CONTROLLER to "RUN" state and immediately starts the motion (according to predefined parameters) of Raster (A or B) or Track modes.

In Slew mode - "G" command does not actually start the motion. This is done by two other "RUN" state subcommands:

"k" - slew forward

"l" - slew reverse

when in "RUN" state, the CONTROLLER will accept two more subcommands:

"Y" - pause

"Q" - continue

these two commands enable to stop and continue motion without exiting the "RUN" state. This is especially useful in Raster (or Sector) mode:

motion can be paused when the current activity is complete, eg. if paused during a scan motion, the positioner will come to rest at the end of that scan and can be continued when required.

If stopped by an "H" and continued by "G" the controller will begin the raster from its initial starting point again! i.e. the mode can only be restarted, not continued after an "H" command.

Note: In all commands affecting motion ("G", "H", "k", "l", "Y", "Q") special acceleration/ deceleration algorithms are incorporated.

3.6 Current Status Query Commands

3.6.1 For one-axis-at-a-time Controllers

Table 3.2: Current Status Commands

ASCII	MNEMONICS	DESCRIPTION
"X1<"	OPR_POS	CURRENT OPERATING (*) AXIS POSITION QUERY
"X2<"	DSPL_POS	CURRENT DISPLAY AXIS (**) (SELECTED BY "E" COMMAND) AXIS POSITION QUERY
"X3<"	VELOCITY	CURRENT VELOCITY QUERY
"X4<"	STATUS	CURRENT SYSTEM STATUS QUERY
"En<" (n = 1-6)	DSPL_SLCT	AXIS SELECTION FOR "X2<" COMMAND

(*) OPERATING AXIS - the axis currently selected in the POWER CONTROL UNIT

(**) DISPLAY AXIS - the right-hand angle display on front panel which has been selected by E command or front panel.



These commands (unlike the MAIN commands) do not have any hierarchical structure. Moreover, to these queries the CONTROLLER will respond whether in "COMPUTER" mode or under panel control (AUTO or MANUAL).

In other words, the "CURRENT STATUS" queries do not change the CONTROLLERS state and are answered at any mode immediately upon reception.

Note: The CONTROLLER's delay between receiving a query and answering it (SRQ going down) is 1 millisecond at most.

The X1 ÷ X4 commands are very useful in monitoring and measuring the positioner motion and locking to a point processes.

Actually using these commands, one can construct on the computer a screen quite similar to the CONTROLLER's front panel:

- * Real time updated left-hand angle display (X1)
- * Real time updated right-hand angle display (X2)
- * Real time updated velocities (X3)
- * Real time updated system status (X4)

So, now, we can modify the "Proper Work Order" flowchart (Fig. 3.3) to the following:

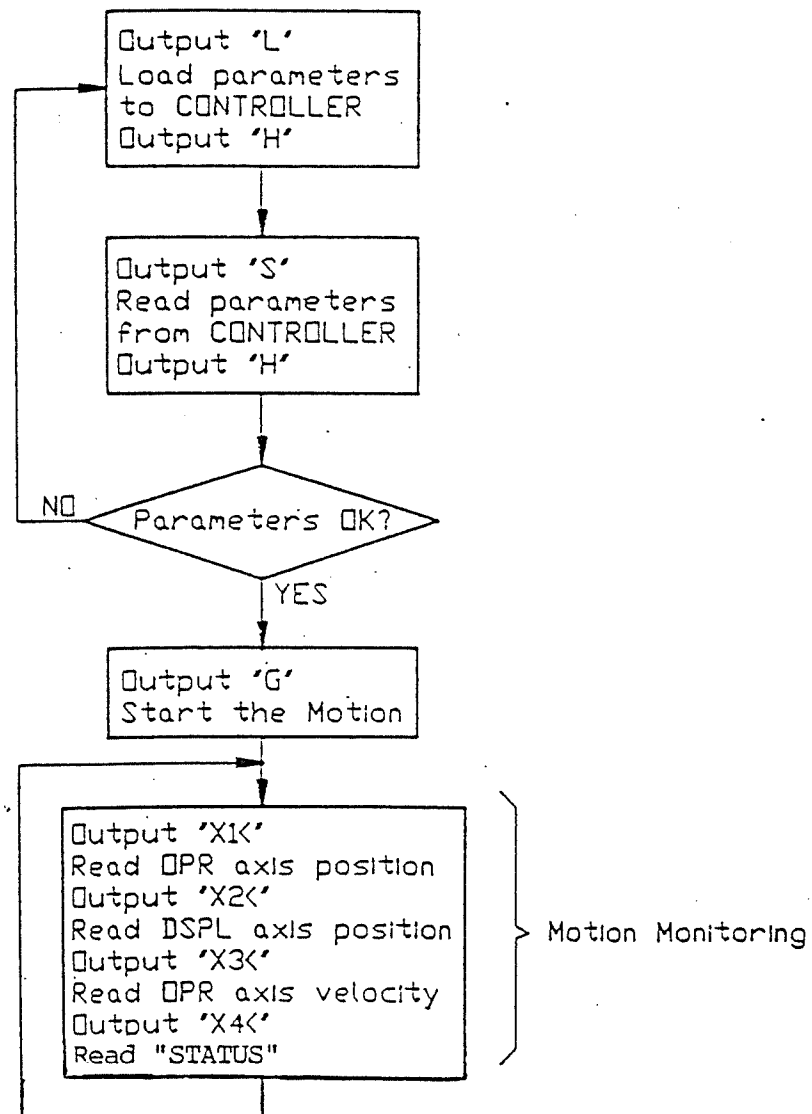


Fig. 3.3: "Proper Order of Work" (with Monitoring)



The controller's responses to the current status queries are in the following formats:

3.6.1.1 Response to "X1<"

The response indicates the current angular position of the OPERATING axis. The response form is the following:

O|X|0|Y|P|z|z|z|z|z|z or O|X|0|Y|P|-|z|z|z|z|z|z|

where:

OX = symbol indicating the OPERATING axis.

y = axis number

P = symbol designating the angular position

- = negative position sign, will appear only in $\pm 180^\circ$ position representation if this particular type is selected (see "Type" selection in para. 3.3.10.6 and also the Operation Manual).

zzzzzz = a string of six integers equivalent to the the requested position in degrees. Had it been written in decimal form, it would have taken the form zzz.zzz.

For instance, if the operating axis is 2 and its current angular position is 132.145° , the response to the command:

"X1<"

is:

O|X|0|2|P|1|3|2|1|4|5

Note the featured resolution of six digits.



3.6.1.2 Response to "X2<"

The response indicates the current angular position of the DISPLAYED axis.

The response form is:

D|X|0|y|P|z|z|z|z|z|z or D|X|0|y|P|-|z|z|z|z|z|z

where:

DX = symbol designating the DISPLAYED axis

y = axis number

p = symbol designating current angular position

- = negative position sign, will appear only in $\pm 180^\circ$ position representation, if this particular type is selected (see "Type" selection in para. 3.3.10.6 and also the Operation Manual).

zzzzzz = string of six integers equivalent to the requested position in degrees. Had it been written in decimal form it would have taken the form zzz.zzz.

For instance, if the DISPLAYED axis is 5 and its current angular position is 15.21° , the response to the following command:

"X2<"

is:

D|X|0|5|P|0|1|5|2|1|0

Note the featured resolution of six digits.

3.6.1.3 Response to "X3<"

The response indicates the current angular velocity of both the OPERATING and DISPLAYED axes. The DISPLAYED axis velocity is, of course, always zero, as the only moving axis is OPERATING. (The only case in which the DISPLAYED axis velocity is not zero is when the DISPLAYED axis is selected to be the same as the OPERATING axis. In such a case, the DISPLAYED axis velocity will be the same as the OPERATING axis.) The reason for this format is compatibility of the COMM LINK for AL-4706-3-2A and AL-4706-3T CONTROLLERS which may operate two simultaneously moving axes. The response form is the following:

O|X|O|Y|V|Z|Z|Z|Z|Z|D|X|O|W|V|K|K|K|K|K

Where:

OX - symbol designating the OPERATING axis

y - OPERATING axis number

DX - symbol designating the DISPLAYED axis

W - DISPLAYED axis number

V - symbol designating current velocity

zzzzz - string of integers equivalent to the OPERATING axis current angular velocity in deg./sec. Had it been written in decimal form it would have taken the form zzz.zz

kkkkk - string of integers equivalent to the DISPLAYED axis current angular velocity in deg./sec. Had it been written in decimal form it would have taken the form: kkk.kk

For instance, if the current velocity of OPERATING axis (axis no. 2) is 9.42 deg./sec. and of the DISPLAYED axis is No. 4, the response to the command "X3<" is:

O|X|O|2|V|0|0|9|4|2|D|X|O|4|V|0|0|0|0|0

Note that these are actual velocities derived from the position sensors. They may differ from the reference values entered to the CONTROLLER in LOAD parameter sequence.

3.6.1.4 Response to "X4<"

The response consists of four consecutive binary (*) bytes named "Status Bytes":

STB1

X	DISPLAYED AXIS			X	OPERATING AXIS		
B17	B16	B15	B14	B13	B12	B11	B10

This byte indicates the serial number of the OPERATING axis and of the additional DISPLAYED axes. These numbers are presented in standard binary notation.

If no axis has been declared OPERATING (for instance, at the initial switch-on of the system), the first three rightmost bits are all zero.

$$B10 = B11 = B12 = 0$$

If no additional axis has been selected by an "E" command for current position readout, the corresponding bits are all 1.

$$B14 = B15 = B16 = 1$$

Bits B13 and B17 are not used.

(*) This is the only (except for the SPOLL BYTE) communication message between the CONTROLLER and computer which is not ASCII characters.



STB2

X	X	X	X	SOFT /HARD	PCU CNTRL	CW LIMIT	CCW LIMIT
B27	B26	B25	B24	B23	B22	B21	B20

B21 B20

0	1	CW Limit tripped
1	0	CCW Limit tripped
1	1	No Limit tripped
0	0	Both Limits tripped

B22 = 1 if system under PCU control

B23 = 0 if hardware limit
1 if software limitSTB3

LOAD	RUN	X	X	READ	X	COMPUTER	STANDBY
B37	B36	B35	B34	B33	B32	B31	B30

B30 = 1 if controller in "STANDBY" state -
0 otherwiseB31 = 1 if communication link is active -
COMPUTER, 0 if it is under panel control

B32 = not used

B33 = 1 if controller is in "READ PARAMETERS" state

B34 = not used

B35 = not used

B36 = 1 if Controller is in RUN state

B37 = 1 if Controller is in "LOAD PARAMETERS" state



STB4

TRACK	RASTER A	SLEW	RASTER B	SLEW CCW	SLEW CW	DIRECT	X
B47	B46	B45	B44	B43	B42	B41	B40

B40 = not used

B41 = 1 if defined motion is DIRECT

B42 = 1 when positioner slews in CW direction

B43 = 1 when positioner slews in CCW direction

B44 = 1 if defined motion is RASTER B

B45 = 1 if defined motion is SLEW

B46 = 1 if defined motion is RASTER A

B47 = 1 if defined motion is TRACK



3.6.2 For two-axes-at-a-time Controllers

Table 3.3: Current Status Commands

ASCII	MNEMONICS	DESCRIPTION
"X1<"	POS_A	CURRENT AXIS A POSITION QUERY
"X2<"	POS_B	CURRENT AXIS B POSITION QUERY
"X3<"	VELOCITY	CURRENT VELOCITY A & B QUERY
"X4<"	STATUS	CURRENT SYSTEM STATUS QUERY



AL-4706-3A COMMUNICATION LINK

These commands (unlike the MAIN commands) do not have any hierarchical structure. Moreover, to these queries the CONTROLLER will respond whether in "COMPUTER" mode or under panel control (AUTO or MANUAL).

In other words, the "CURRENT STATUS" queries do not change the CONTROLLERS state and are answered at any mode immediately upon reception.

Note: The CONTROLLER's delay between receiving a query and answering it (SRQ going down) is 1 millisecond at most.

The X1 ÷ X4 commands are very useful in monitoring and measuring the positioner motion and locking to a point processes.

Actually using these commands, one can construct on the computer a screen quite similar to the CONTROLLER's front panel:

- * Real time updated Axis A angle display (X1)
- * Real time updated Axis B angle display (X2)
- * Real time updated velocities (X3)
- * Real time updated system status (X4)

So, now, we can modify the "Proper Work Order" flowchart (Fig. 3.4) to the following:

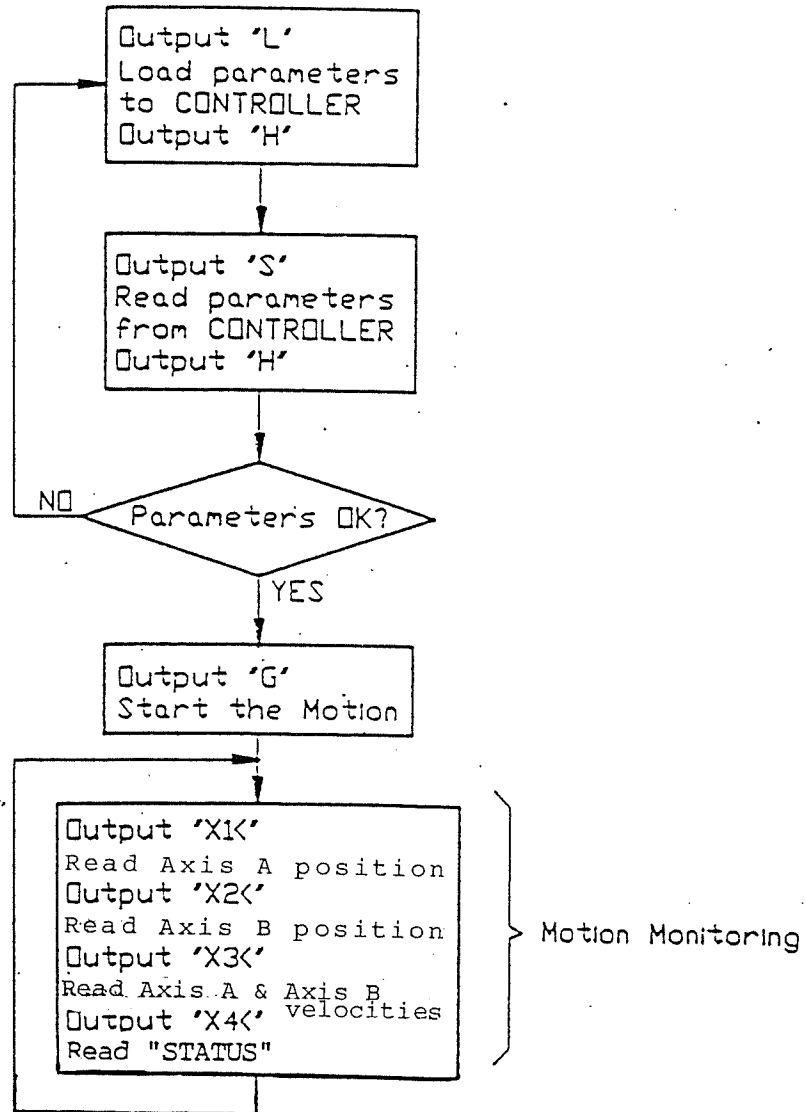


Fig. 3.4: "Proper Order of Work" (with Monitoring)



The controller's responses to the current status queries are in the following formats:

3.6.2.1 Response to "X1<"

The response indicates the current angular position of the Axis A axis. The response form is the following:

O|X|O|Y|P|z|z|z|z|z|z or O|X|O|Y|P|-|z|z|z|z|z|z|

where:

OX = symbol indicating the Axis A.

y = axis number

P = symbol designating the angular position

- = negative position sign, will appear only in $\pm 180^\circ$ position representation if this particular type is selected (see "Type" selection in para. 3.3.10.6 and also the Operation Manual).

zzzzzz = a string of six integers equivalent to the the requested position in degrees. Had it been written in decimal form, it would have taken the form zzz.zzz.

For instance, if the Axis A is 2 and its current angular position is 132.145° , the response to the command:

"X1<"

is:

O|X|O|2|P|1|3|2|1|4|5

Note the featured resolution of six digits.



3.6.2.2 Response to "X2<"

The response indicates the current angular position of the Axis B.

The response form is:

D|X|0|Y|P|z|z|z|z|z|z or D|X|0|Y|P|-|z|z|z|z|z|z

where:

DX = symbol designating the Axis B.

y = axis number

p = symbol designating current angular position

- = negative position sign, will appear only in $\pm 180^\circ$ position representation, if this particular type is selected (see "Type" selection in para. 3.3.10.6 and also the Operation Manual).

zzzzzz = string of six integers equivalent to the requested position in degrees. Had it been written in decimal form it would have taken the form zzz.zzz.

For instance, if the Axis B is 5 and its current angular position is 15.21° , the response to the following command:

"X2<"

is:

D|X|0|5|P|0|1|5|2|1|0

Note the featured resolution of six digits.



3.6.2.3 Response to "X3<"

The response indicates the current angular velocity of both the A & B axes.

O|X|0|Y|V|Z|Z|Z|Z|Z|D|X|0|W|V|K|K|K|K|K

Where:

OX - symbol designating the Axis A

y - Axis A axis number

DX - symbol designating the Axis B.

W - Axis B axis number

V - symbol designating current velocity

zzzzz - string of integers equivalent to the Axis A current angular velocity in deg./sec. Had it been written in decimal form it would have taken the form zzz.zz

kkkkk - string of integers equivalent to the Axis B axis current angular velocity in deg./sec. Had it been written in decimal form it would have taken the form: kkk.kk

For instance, if the current velocities of Axis A (No. 2) is 9.42 deg./sec. and Axis B (No. 4) is 5.76 deg/sec, the response to the command "X3<" is:

O|X|0|2|V|0|0|9|4|2|D|X|0|4|V|0|0|5|7|6

Note that these are actual velocities derived from the position sensors. They may differ from the reference values entered to the CONTROLLER in LOAD parameter sequence.



3.6.2.4 Response to "X4<"

The response consists of four consecutive binary (*) bytes named "Status Bytes":

STB1

X	AXIS B			X	AXIS A		
B17	B16	B15	B14	B13	B12	B11	B10

The axes numbers are presented in standard binary notation.

If no axis has been declared Axis A (for instance, at the initial switch-on of the system), the first three rightmost bits are all zero.

$$B10 = B11 = B12 = 0$$

If no axis has been declared Axis B, the corresponding bits are all 1.

$$B14 = B15 = B16 = 1$$

Bits B13 and B17 are not used.

(*) This is the only (except for the SPOLL BYTE) communication message between the CONTROLLER and computer which is not ASCII characters.



STB2

AXIS B				AXIS A			
SOFT/ HARD	PCU CNTRL	CW LIMIT	CCW LIMIT	SOFT/ HARD	PCU CNTRL	CW LIMIT	CCW LIMIT
B27	B26	B25	B24	B23	B22	B21	B20

for Axis A:

B21 B20

0	1	CW Limit tripped
1	0	CCW Limit tripped
1	1	No Limit tripped
0	0	Both Limits tripped

B22 = 1 if system under PCU control

B23 = 0 if hardware limit
1 if software limit

for Axis B:

B25 B24

0	1	CW Limit tripped
1	0	CCW Limit tripped
1	1	No Limit tripped
0	0	Both Limits tripped

B26 = 1 if system under PCU control

B27 = 0 if hardware limit
1 if software limit

STB3

LOAD	RUN	X	X	READ	X	COMPUTER	STANDBY
B37	B36	B35	B34	B33	B32	B31	B30

B30 = 1 if controller in "STANDBY" state -
0 otherwise

B31 = 1 if communication link is active -
COMPUTER, 0 if it is under panel control

B32 = not used

B33 = 1 if controller is in "READ PARAMETERS" state

B34 = not used

B35 = not used

B36 = 1 if Controller is in RUN state

B37 = 1 if Controller is in "LOAD PARAMETERS" state

STB4

TRACK	RASTER A	SLEW	RASTER B	SLEW CCW	SLEW CW	DIRECT	X
B47	B46	B45	B44	B43	B42	B41	B40

B40 = not used

B41 = 1 if defined motion is DIRECT

B42 = 1 when positioner slews in CW direction

B43 = 1 when positioner slews in CCW direction

B44 = 1 if defined motion is RASTER B

B45 = 1 if defined motion is SLEW

B46 = 1 if defined motion is RASTER A

B47 = 1 if defined motion is TRACK



CHAPTER 4

"COMM LINK" Commands Sequences

4.1 Scope

This chapter demonstrates some of more commonly used "COMM LINK" commands sequences.

It also describes how such commands can be "chained" into one message terminated by "CR", "LF". The sequences illustrated use BASIC 4.0 (or later) language employed on HP 300 series computer. This does not affect the general nature of the discussion when applied to other computer system.

(Although the IEEE-488 interfacing may require extra program lines: note that in HP Basic, an OUTPUT statement activates automatically all the IEEE-488 controls and is also terminated by a "CR", "LF"; same remarks apply to the ENTER statement).



4.2 Chaining the "COMM LINK" Commands

The "COMM LINK" commands may be chained together in one message terminated by an ASCII "CR", "LF" sequence, if the two following rules are maintained:

- * The chained commands are not queries (neither "READ PARAMETERS" nor "Current Status").
- * The commands chain isn't longer than 60 characters.
- * "G" command cannot be chained after "H" command, as there must be a delay of at least 10msec between the two commands (see para. 3.2).



AL-4706-3A COMMUNICATION LINK

4.2.1 Ex. No. 1



```
10  ! Setting up a Track to Target movement
20  !
30  ! Example No:1
40  !
50  !
60  !*****
70  !*          ***  W A R N I N G  ***          *
80  !*      Be sure to adapt the loaded parameters to your system:      *
90  !*      - axis selection, according to the connected axes            *
100 !*      - position targets, according to the limit switches          *
110 !*      Assure that the configuration parameters are set correctly    *
120 !*****
130 !
140 !
150 Device=704                      !CONTROLLER IEEE-488 ADDRESS
160 !
170 OUTPUT Device;"H"              !Turn CONTROLLER to "STAND-BY" State
180 !
190 OUTPUT Device;"L"              !Turn CONTROLLER to "LOAD" State
200 !
210 OUTPUT Device;"Aa1<"          !Select first rotation axis No:1
220 OUTPUT Device;"Ae3<"          !Select second rotation axis No:3
230 OUTPUT Device;"Pat05000<"     !Set target of first axis :50.00 deg
240 OUTPUT Device;"Pet35180<"     !Set target of first axis :351.8 deg
250 OUTPUT Device;"Daf<"          !Set direction of first axis :forward
260 OUTPUT Device;"Der<"          !Set direction of second axis :reverse
270 OUTPUT Device;"Va00876<"      !Set travel velocity of axis a:87.6%
280 OUTPUT Device;"Ve00564<"      !Set travel velocity of axis e:56.4%
290 OUTPUT Device;"MT"            !Set the motion mode:Track
300 !
310 OUTPUT Device;"H"              !Return to "STAND-BY"
320 !
330 WAIT .010                      !Wait 10 msec between 'H' & 'G'
340 !
350 OUTPUT Device;"G"              !Turn CONTROLLER to "RUN":Start motion
360 !
370 END
```



4.2.2 Ex. No. 2



```
10  !   Setting up a Track to Target movement
20  !
30  !   Example No:2
40  !
50  !*****
60  !*               ***   W A R N I N G   ***
70  !*   Be sure to adapt the loaded parameters to your system:
80  !*   - axis selection, according to the connected axes
90  !*   - position targets, according to the limit switches
100 !*   Assure that the configuration parameters are set correctly
110 !*****
120 !
130 !
140 Device=704                !CONTROLLER IEEE-488 ADDRESS
150 !
160 OUTPUT Device;"H"         !Turn CONTROLLER to "STAND-BY" State
170 !
180 OUTPUT Device;"L"         !Turn CONTROLLER to "LOAD" State
190 !
200 OUTPUT Device;"Aa1<Ae3<"    !Select first rotation axis No:1, and
210                               !...select second rotation axis No:3
220 OUTPUT Device;"Pat05000<Pet35180<"    !Set first & second axes
230                               !targets :50deg & 351.8deg
240 OUTPUT Device;"Daf<Der<"    !Set direction of first axis :forward,
250                               !... direction of second axis :reverse
260 OUTPUT Device;"Va00876<Ve00564<"    !Set velocity of axis a:87.6%,
270                               !and velocity of axis e:56.4%
280 OUTPUT Device;"MTH"         !Set the motion mode:Track, & ...
290                               !           ...Return to "STAND-BY"
300 WAIT .010                  !Wait 10 msec between 'H' & 'G'
310 !
320 OUTPUT Device;"G"          !Start motion
330 !
340 END
```



4.2.3 Ex. No. 3



```
10  !   Setting up a Track to Target movement
20  !
30  !   Example No:3
40  !
50  !
60  !*****
70  !*           ***   W A R N I N G   ***           *
80  !*   Be sure to adapt the loaded parameters to your system:   *
90  !*   - axis selection, according to the connected axes         *
100 !*   - position targets, according to the limit switches       *
110 !*   Assure that the configuration parameters are set correctly *
120 !*****
130 !
140 !
150 Device=704                      !CONTROLLER IEEE-488 ADDRESS
160 !
170 OUTPUT Device;"HLAa1<Ae3<Pat05000<Pet35180<Daf<Der<Ve00876<MTH"
180 !
190 WAIT .010
200 !
210 OUTPUT Device;"G"
220 !
230 END
```



4.3 Setting up different motions - LOADING & RUNNING only

Herein are included the most simple setups for all the available motion modes:

Track, Raster A, Raster B, Sector, Slew.

These setups, just output all the necessary parameters to the Controller and then initiate the "RUN" command.



4.3.1 Setting Up a Track to Target Motion

11



```
10  !   Setting up a Track to Target movement
20  !
30  !
40  !
50  !
60  !*****
70  !*                               ***  W A R N I N G  ***                               *
80  !*   Be sure to adapt the loaded parameters to your system:                               *
90  !*   - axis selection, according to the connected axes                               *
100 !*   - position targets, according to the limit switches                               *
110 !*   Assure that the configuration parameters are set correctly                               *
120 !*****
130 !
140 !
150 Device=704                               !CONTROLLER IEEE-488 ADDRESS
160 !
170 OUTPUT Device;"H"                               !Turn CONTROLLER to "STAND-BY" State
180 !
190 OUTPUT Device;"L"                               !Turn CONTROLLER to "LOAD" State
200 !
210 OUTPUT Device;"Aa1<"                               !Select first rotation axis No:1
220 OUTPUT Device;"Aa3<"                               !Select second rotation axis No:3
230 OUTPUT Device;"Pat05000<"                               !Set target of first axis :50.00 deg
240 OUTPUT Device;"Pet35180<"                               !Set target of first axis :351.8 deg
250 OUTPUT Device;"Daf<"                               !Set direction of first axis :forward
260 OUTPUT Device;"Der<"                               !Set direction of second axis :reverse
270 OUTPUT Device;"Va00876<"                               !Set travel velocity of axis a:87.6%
280 OUTPUT Device;"Ve00564<"                               !Set travel velocity of axis e:56.4%
290 OUTPUT Device;"MT"                               !Set the motion mode:Track
300 !
310 OUTPUT Device;"H"                               !Return to "STAND-BY"
320 !
330 WAIT .010                               !Wait 10 msec between 'H' & 'G'
340 !
350 OUTPUT Device;"G"                               !Turn CONTROLLER to "RUN":Start motion
360 !
370 END
```



4.3.2 Setting Up a RASTER SCAN (A) Motion

Note: The "RASTER" scan in the following example will be executed either in "forward" or "reverse" direction, according to the specified direction parameter: "Daf<" or "Dar<" (not included in the program).



```
10      ! Setting up a RASTER SCAN (A) movement
20      !
30      !
40      !*****
50      !*                ***  W A R N I N G  ***                *
60      !*      Be sure to adapt the loaded parameters to your system:      *
70      !*      - axis selection, according to the connected axes            *
80      !*      - position targets, according to the limit switches          *
90      !*      Assure that the configuration parameters are set correctly    *
100     !*****
110     !
120     !
130     Device=704                      !CONTROLLER IEEE-488 ADDRESS
140     !
150     OUTPUT Device;"H"                !Turn CONTROLLER to "STAND-BY" State
160     !
170     OUTPUT Device;"L"                !Turn CONTROLLER to "LOAD" State
180     !
190     OUTPUT Device;"Aa2<"            !Select scan axis No:2
200     OUTPUT Device;"Aa3<"            !Select step axis No:3
210     OUTPUT Device;"Pas00000<"       !Set scan starting point:0 deg
220     OUTPUT Device;"Pab11111<"       !Set scan ending point:111.11 deg
230     OUTPUT Device;"Pes00000<"       !Set step starting point:0 deg
240     OUTPUT Device;"Peb05470<"       !Set step ending point:54.7 deg
250     OUTPUT Device;"J00500<"         !Set step value:5 deg
260     OUTPUT Device;"I00000<"         !Switch off the increments
270     OUTPUT Device;"Va00736<"        !Set scan velocity :75.6%
280     OUTPUT Device;"Ve00559<"        !Set step velocity :55.9%
290     OUTPUT Device;"MR"              !Set the motion mode:Raster_A
300     !
310     OUTPUT Device;"H"                !Return to "STAND-BY"
320     !
330     WAIT .010                       !Wait 10msec between 'H' & 'G'
340     !
350     OUTPUT Device;"G"                !Turn CONTROLLER to "RUN":Start motion
360     !
370     END
```



4.3.3 Setting Up a RASTER SCAN (B) Motion

Note: The "RASTER" scan in the following example will be executed either in "forward" or "reverse" direction, according to the specified direction parameter: "Daf<" or "Dar<" (not included in the program).



```
10  ! Setting up a FASTER SCAN (B) movement
20  !
30  !
40  !*****
50  !*          ***  W A R N I N G  ***          *
60  !*          Be sure to adapt the loaded parameters to your system:          *
70  !*          - axis selection, according to the connected axes                *
80  !*          - position targets, according to the limit switches              *
90  !*          Assure that the configuration parameters are set correctly        *
100 !*****
110 !
120 !
130 Device=704                      !CONTROLLER IEEE-468 ADDRESS
140 !
150 OUTPUT Device;"H"              !Turn CONTROLLER to "STAND-BY" State
160 !
170 OUTPUT Device;"L"              !Turn CONTROLLER to "LOAD" State
180 !
190 OUTPUT Device;"Aa2<"           !Select scan axis No:2
200 OUTPUT Device;"Ae3<"           !Select step axis No:3
210 OUTPUT Device;"Pas00000<"      !Set scan starting point:0 deg
220 OUTPUT Device;"Pab11111<"      !Set scan ending point:111.11 deg
230 OUTPUT Device;"Pes00000<"      !Set step starting point:0 deg
240 OUTPUT Device;"Peb05470<"      !Set step ending point:54.7 deg
250 OUTPUT Device;"J00500<"        !Set step value:5 deg
260 OUTPUT Device;"I00000<"        !Switch off the increments
270 OUTPUT Device;"Va00756<"        !Set scan velocity :75.6%
280 OUTPUT Device;"Ve00559<"        !Set step velocity :55.9%
290 OUTPUT Device;"MN"             !Set the motion mode:Raster_B
300 !
310 OUTPUT Device;"H"              !Return to "STAND-BY"
320 !
330 WAIT .010                      !Wait 10msec between 'H' & 'G'
340 !
350 OUTPUT Device;"G"              !Turn CONTROLLER to "RUN":Start motion
360 !
370 END
```



4.3.4 Setting Up a SECTOR SCAN Motion

Note: The "SECTOR" scan in the following example will be executed either in "forward" or "reverse" direction, according to the specified direction parameter: "Daf<" or "Dar<" (not included in the program).



```
10  !   Setting up a SECTOR SCAN  movement
20  !
30  !
40  !*****
50  !*           ***   W A R N I N G   ***           *
60  !*   Be sure to adapt the loaded parameters to your system:   *
70  !*   - axis selection, according to the connected axes         *
80  !*   - position targets, according to the limit switches       *
90  !*   Assure that the configuration parameters are set correctly *
100 !*****
110 !
120 !
130 !   The TTL Increment Pulses may be seen on a oscilloscope connected
140 !   to J1 connector on the rear panel
150 !
160 !
170 Device=704                !CONTROLLER IEEE-488 ADDRESS
180 !
190 OUTPUT Device;"H"         !Turn CONTROLLER to "STAND-BY" State
200 !
210 OUTPUT Device;"L"         !Turn CONTROLLER to "LOAD" State
220 !
230 OUTPUT Device;"Aa2<"      !Select scan axis No:2
240 OUTPUT Device;"Ae0<"      !Select step axis No:0
250 OUTPUT Device;"Fas03344<" !Set scan starting point:33.44 deg
260 OUTPUT Device;"Fab21212<" !Set scan ending point:212.12 deg
270 OUTPUT Device;"J00007<"   !Set number of scans: 7
280 OUTPUT Device;"I00050<"   !Increments Gap:.05 deg
290 OUTPUT Device;"Fz<"       !Disable GPIB Increments
300 OUTPUT Device;"Va00500<"  !Set scan velocity :half speed
310 OUTPUT Device;"MN"        !Set the motion mode:Raster_B (could be A)
320 !
330 OUTPUT Device;"H"         !Return to "STAND-BY"
340 !
350 WAIT .010                 ! Wait 10msec between 'H' & 'G'
360 !
370 OUTPUT Device;"G"         !Turn CONTROLLER to "RUN":Start motion
380 !
390 END
```



4.3.5 Setting Up a SLEW Motion



```
10 ! Setting up a SLEW movement
20 !
30 ! This program toggles slew direction by the "ENTER" key
40 !
50 !
60 !*****
70 !*          ***  W A R N I N G  ***          *
80 !*      Be sure to adapt the loaded parameters to your system:      *
90 !*      - axis selection, according to the connected axes            *
100 !*      - position targets, according to the limit switches          *
110 !*      Assure that the configuration parameters are set correctly    *
120 !*****
130 !
140 !
150 Device=704                      !CONTROLLER IEEE-488 ADDRESS
160 !
170 OUTPUT Device;"H"              !Turn CONTROLLER to "STAND-BY" State
180 !
190 OUTPUT Device;"L"              !Turn CONTROLLER to "LOAD" State
200 !
210 OUTPUT Device;"Az3<"          !Select slew axis No:3
220 OUTPUT Device;"Vz00459<"      !Set slew velocity :45.9%
230 OUTPUT Device;"MZ"            !Set the motion mode:Slew
240 !
250 OUTPUT Device;"H"              !Return to "STAND-BY"
260 !
270 WAIT .010                      !Wait 10msec between "H" & "G"
280 !
290 OUTPUT Device;"G"              !Turn CONTROLLER to "RUN"
300 !
310 Ff=0
320 !
330 Again: INPUT "Press ENTER to toggle direction",A$
340 Ff=NOT (Ff)
350 IF Ff=0 THEN
360     BEEP 300,.3
370     OUTPUT Device;"k"
380     PRINT "FORWARD"
390 END IF
400 IF Ff=1 THEN
410     BEEP 900,.3
420     OUTPUT Device;"l"
430     PRINT "REVERSE"
440 END IF
450 GOTO Again
460 !
470 END
```



4.4 ENTERING DATA Routines

In order to make more sophisticated programs than the ones in Para. 4.3 one must learn to read data from the CONTROLLER.

To illustrate this, the next three subroutines are presented. These subroutines complement the three data-entering options describes in Chapter 2:

- PPOLL
- SPOLL
- SRQ INTERRUPT



4.4.1 Entering DATA Using PPOLL Method



```
10  !*****
20  !*          This Subroutine enters data from          *
30  !*          AL-4706-3 TYPE  CONTROLLERS              *
40  !*          via IEEE-488 bus using PPOLL method      *
50  !*****
60  !
70  !
80          SUB Enter_ppoll
90          !
100         COM /Ieee_addr/ Device,Hpib
110         COM /In_data/ Data$(20)
120         COM /Flag/ Newparam,Answ_x1,Answ_x2,Answ_x3,Answ_x4
130         COM /Incr_dspl/ X,Y,Tot_count,Prv_incr$(15),Prv_x,Prv_y
140         !
150 Ppoll:   P=PPOLL(Hpib)
160         SELECT 1
170         CASE BIT(P,3)
180             GOSUB Increment
190         CASE BIT(P,4)
200             GOTO Spoll
210         END SELECT
220         GOTO Sub_end
230         !
240 Spoll:   S=SPOLL(Device)
250         SELECT 1
260         CASE BIT(S,7)
270             GOTO Answer !Answer on "READ" or "Curr_Stat" queries
280         CASE NOT BIT(S,3) AND NOT BIT(S,2) AND NOT BIT(S,1) AND NOT BIT
T(S,0)
290             BEEP 300,.2
300             PRINT TABXY(60,16),CHR$(137);"BOTH LIMITS!"
310             PRINT TABXY(60,17)," SPOLL=";S
320         CASE NOT BIT(S,3) AND NOT BIT(S,2) AND NOT BIT(S,1) AND BIT(S,
0)
330             BEEP 300,.2
340             PRINT TABXY(60,16),CHR$(137);"**CW LIMIT**"
350             PRINT TABXY(60,17)," SPOLL=";S
360         CASE NOT BIT(S,3) AND NOT BIT(S,2) AND BIT(S,1) AND NOT BIT(S,
0)
370             BEEP 300,.2
380             PRINT TABXY(60,16),CHR$(137);"**CCW LIMIT**"
390             PRINT TABXY(60,17)," SPOLL=";S
400         CASE NOT BIT(S,3) AND NOT BIT(S,2) AND BIT(S,1) AND BIT(S,0)
410             BEEP 300,.2
420             PRINT TABXY(60,16),CHR$(137);" PULLED OUT "
430             PRINT TABXY(60,17)," SPOLL=";S
440         CASE NOT BIT(S,3) AND BIT(S,2) AND NOT BIT(S,1) AND NOT BIT(S,
0)
450             Err_flg=1
460             BEEP 300,.2
470             PRINT TABXY(60,16),CHR$(137);"** ERROR! **"
480             PRINT TABXY(60,17)," SPOLL=";S
```



```
490      CASE BIT(S,3) AND BIT(S,2) AND NOT BIT(S,1) AND NOT BIT(S,0)
500          PRINT TABXY(46,19),CHR$(129);Tot_count;CHR$(128)
510          Tot_count=Tot_count+1
520      CASE BIT(S,3) AND BIT(S,2) AND BIT(S,1) AND NOT BIT(S,0)
530          BEEP 300,.2
540          PRINT TABXY(60,16),CHR$(137);"** FAILED **"
550          PRINT TABXY(60,17)," SPOLL=";S
560      CASE BIT(S,3) AND BIT(S,2) AND BIT(S,1) AND BIT(S,0)
570          BEEP 300,.2
580          PRINT TABXY(60,16),CHR$(137);"** PASSED **"
590          PRINT TABXY(60,17)," SPOLL=";S
600      CASE BIT(S,3) AND NOT BIT(S,2) AND NOT BIT(S,1) AND NOT BIT(S,
0)
610          BEEP 300,.2
620          PRINT TABXY(60,16),CHR$(137);"** LOCKED **"
630          PRINT TABXY(60,17)," SPOLL=";S
640      CASE ELSE
650          BEEP 300,.2
660          PRINT TABXY(60,16),CHR$(137);" UNRESOLVED "
670          PRINT TABXY(60,17)," SPOLL=";S
680      END SELECT
690      PRINT TABXY(1,1),CHR$(139)
700      GOTO Sub_end
710      !
720 Answer: ENTER Device;Data$
730          SELECT Data$[1,1]
740          CASE "0"
750              SELECT Data$[5,5]
760              CASE "P"                      !Answer on "X1<" query:
770                  IF Data$[6,6]="-" THEN
780                      PRINT TABXY(13,7),Data$[6,9];". ";Data$[10];" deg"
790                  ELSE
800                      PRINT TABXY(13,7),Data$[6,8];". ";Data$[9];" deg "
810                  END IF
820                  PRINT TABXY(13,5),Data$[4,4]          ! OPR axis number
830                  Answ_x1=1
840              CASE "V"                      !Answer on "X3<" query:
850                  PRINT TABXY(13,9),Data$[6,8];". ";Data$[9,10];" deg/sec"
860                  IF Contr_type$="4706-3-2A" THEN
870                      PRINT TABXY(39,9),Data$[16,18];". ";Data$[19];" deg/sec"
880                  END IF
890                  Answ_x3=1
900              END SELECT
910          CASE "D"                      !Answer on "X2<" query:
920              IF Data$[6,6]="-" THEN
930                  PRINT TABXY(39,7),Data$[6,9];". ";Data$[10];" deg"
940              ELSE
950                  PRINT TABXY(39,7),Data$[6,8];". ";Data$[9];" deg "
960              END IF
970              PRINT TABXY(39,5),Data$[4,4]          ! DSPL axis number
980              Answ_x2=1
```



```
990          CASE "S"                                !Answer on "READ" query
1000              Newparam=1
1010              GOTO Sub_end
1020          CASE ELSE
1030              GOTO System_stat                    !Answer on "X4<" query
1040          END SELECT
1050          GOTO Sub_end
1060          !
1070 System_stat: !Answ_x4=1                          "X4<" query not in use
1080          GOTO Sub_end
1090          !
1100 Increment: ENTER Device;Incr$
1110          IF Incr$[1,1]<>"-" THEN Incr$=" "&Incr$
1120          PRINT TABXY(X,Y);CHR$(129);Incr$
1130          PRINT TABXY(46,19),Tot_count;CHR$(128)
1140          IF Tot_count-1>0 THEN PRINT TABXY(Prv_x,Prv_y),Prv_incr$
1150          Tot_count=Tot_count+1
1160          Prv_x=X
1170          Prv_y=Y
1180          Prv_incr$=Incr$
1190          X=X+7
1200          IF X>42 THEN
1210              X=3
1220              Y=Y+1
1230              IF Y>19 THEN Y=13
1240          END IF
1250          RETURN
1260          !
1270          !
1280          !*****
1290          !*          END          of          PPOLL-Enter          Routine          *
1300          !*****
1310          !
1320 Sub_end:   SUBEND
```



4.4.2 Entering DATA Using SPOLL Method



```
10 *****
20 !*          This Subroutine enters data from          *
30 !*          AL-4706-3 TYPE  CONTROLLERS              *
40 !*          via IEEE-488 bus using SPOLL method      *
50 *****
60 !
70 !
80          SUB Enter_spoll
90          !
100         COM /Ieee_addr/ Device,Hpib
110         COM /In_data/ Data$(20)
120         COM /Flag/ Newparam,Answ_x1,Answ_x2,Answ_x3,Answ_x4
130         COM /Incr_dsp1/ X,Y,Tot_count,Prv_incr$(15),Prv_x,Prv_y
140         !
150 Spoll:   S=SPOLL(Device)
160         SELECT 1
170         CASE NOT BIT(S,6)
180             GOTO Sub_end
190         CASE BIT(S,7)
200             GOTO Answer !Answer on "READ" or "Curr_Stat" queries
210         CASE NOT BIT(S,3) AND NOT BIT(S,2) AND NOT BIT(S,1) AND NOT BIT(S,0)
220             BEEP 300,.2
230             PRINT TABXY(60,16),CHR$(137);"BOTH LIMITS!"
240             PRINT TABXY(60,17)," SPOLL=";S
250         CASE NOT BIT(S,3) AND NOT BIT(S,2) AND NOT BIT(S,1) AND BIT(S,0)
260             BEEP 300,.2
270             PRINT TABXY(60,16),CHR$(137);"**CW LIMIT**"
280             PRINT TABXY(60,17)," SPOLL=";S
290         CASE NOT BIT(S,3) AND NOT BIT(S,2) AND BIT(S,1) AND NOT BIT(S,0)
300             BEEP 300,.2
310             PRINT TABXY(60,16),CHR$(137);"**CCW LIMIT**"
320             PRINT TABXY(60,17)," SPOLL=";S
330         CASE NOT BIT(S,3) AND NOT BIT(S,2) AND BIT(S,1) AND BIT(S,0)
340             BEEP 300,.2
350             PRINT TABXY(60,16),CHR$(137);" PULLED OUT "
360             PRINT TABXY(60,17)," SPOLL=";S
370         CASE NOT BIT(S,3) AND BIT(S,2) AND NOT BIT(S,1) AND NOT BIT(S,0)
380             Err_flg=1
390             BEEP 300,.2
400             PRINT TABXY(60,16),CHR$(137);"** ERROR! **"
410             PRINT TABXY(60,17)," SPOLL=";S
420         CASE BIT(S,3) AND BIT(S,2) AND BIT(S,1) AND NOT BIT(S,0)
430             BEEP 300,.2
440             PRINT TABXY(60,16),CHR$(137);"** FAILED **"
450             PRINT TABXY(60,17)," SPOLL=";S
460         CASE BIT(S,3) AND BIT(S,2) AND NOT BIT(S,1) AND BIT(S,0)
470             PRINT TABXY(46,19),CHR$(129);Tot_count;CHR$(128)
480             Tot_count=Tot_count+1
```



```
490      CASE BIT(S,3) AND BIT(S,2) AND BIT(S,1) AND BIT(S,0)
500          BEEP 300,.2
510          PRINT TABXY(60,16),CHR$(137);"** PASSED **"
520          PRINT TABXY(60,17)," S POLL=";S
530      CASE BIT(S,3) AND NOT BIT(S,2) AND NOT BIT(S,1) AND NOT BIT(S,
0)
540          BEEP 300,.2
550          PRINT TABXY(60,16),CHR$(137);"** LOCKED **"
560          PRINT TABXY(60,17)," S POLL=";S
570      CASE ELSE
580          BEEP 300,.2
590          PRINT TABXY(60,16),CHR$(137);" UNRESOLVED "
600          PRINT TABXY(60,17)," S POLL=";S
610      END SELECT
620      PRINT TABXY(1,1),CHR$(139)
630      GOTO Sub_end
640      !
650 Answer: ENTER Device;Data$
660      IF LEN(Data$)=6 AND Data$[1,1]<>"S" OR Data$[1,1]="-" THEN GOT
0 Increment
670      SELECT Data$[1,1]
680      CASE "0"
690          SELECT Data$[5,5]
700          CASE "P"                                !Answer on "X1<" query:
710              IF Data$[6,6]="-" THEN
720                  PRINT TABXY(13,7),Data$[6,9];".";Data$[10];" deg"
730              ELSE
740                  PRINT TABXY(13,7),Data$[6,8];".";Data$[9];" deg "
750              END IF
760              PRINT TABXY(13,5),Data$[4,4]          ! OPR axis number
770              Answ_x1=1
780          CASE "V"                                !Answer on "X3<" query:
790              PRINT TABXY(13,9),Data$[6,8];".";Data$[9,10];" deg/sec"
800              IF Contr_type$="4706-3-2A" THEN
810                  PRINT TABXY(39,9),Data$[16,18];".";Data$[19];" deg/sec"
820              END IF
830              Answ_x3=1
840          END SELECT
850      CASE "D"                                !Answer on "X2<" query:
860          IF Data$[6,6]="-" THEN
870              PRINT TABXY(39,7),Data$[6,9];".";Data$[10];" deg"
880          ELSE
890              PRINT TABXY(39,7),Data$[6,8];".";Data$[9];" deg "
900          END IF
910          PRINT TABXY(39,5),Data$[4,4]              ! DSPL axis number
920          Answ_x2=1
930      CASE "S"                                !Answer on "READ" query
940          Newparam=1
950          GOTO Sub_end
960      CASE ELSE
970          GOTO System_stat                        !Answer on "X4<" query
980      END SELECT
990      GOTO Sub_end
```



```
1000      !
1010 System_stat: !Answ_x4=1                      "X4<" query not in use
1020      GOTO Sub_end
1030      !
1040 Increment:  IF Data$(1,1)<>"-" THEN Data$=" "&Data$
1050      PRINT TABXY(X,Y);CHR$(129);Data$
1060      PRINT TABXY(46,19),Tot_count;CHR$(128)
1070      IF Tot_count-1>0 THEN PRINT TABXY(Prv_x,Prv_y),Prv_incr$
1080      Tot_count=Tot_count+1
1090      Prv_x=X
1100      Prv_y=Y
1110      Prv_incr$=Data$
1120      X=X+7
1130      IF X>42 THEN
1140          X=3
1150          Y=Y+1
1160          IF Y>19 THEN Y=13
1170      END IF
1180      !
1190      !
1200      !*****
1210      !*          END      of      SPOLL-Enter      Routine      *
1220      !*****
1230      !
1240 Sub_end:  SUBEND
```



4.4.3 Entering DATA Using SRQ INTERRUPT Method



```
10  !*****
20  !*      This Subroutine enters data from      *
30  !*      AL-4706-3 TYPE  CONTROLLERS          *
40  !*      via IEEE-488 bus using SRQ-INTR method *
50  !*****
60  !
70  !
80      SUB Enter_intr
90      !
100     COM /Ieee_addr/ Device,Hpib
110     COM /In_data/ Data$(20)
120     COM /Flag/ Newparam,Answ_x1,Answ_x2,Answ_x3,Answ_x4
130     COM /Incr_dspl/ X,Y,Tot_count,Prv_incr$(15),Prv_x,Prv_y
140     !
150  Spoll: S=SPOLL(Device)
160     SELECT 1
170     CASE BIT(S,7)
180         GOTO Answer !Answer on "READ" or "Curr_Stat" queries
190     CASE NOT BIT(S,3) AND NOT BIT(S,2) AND NOT BIT(S,1) AND NOT BIT(S,0)
200         BEEP 300,.2
210         PRINT TABXY(60,16),CHR$(137);"BOTH LIMITS!"
220         PRINT TABXY(60,17)," SPOLL=";S
230     CASE NOT BIT(S,3) AND NOT BIT(S,2) AND NOT BIT(S,1) AND BIT(S,0)
240         BEEP 300,.2
250         PRINT TABXY(60,16),CHR$(137);"**CW LIMIT**"
260         PRINT TABXY(60,17)," SPOLL=";S
270     CASE NOT BIT(S,3) AND NOT BIT(S,2) AND BIT(S,1) AND NOT BIT(S,0)
280         BEEP 300,.2
290         PRINT TABXY(60,16),CHR$(137);"**CCW LIMIT**"
300         PRINT TABXY(60,17)," SPOLL=";S
310     CASE NOT BIT(S,3) AND NOT BIT(S,2) AND BIT(S,1) AND BIT(S,0)
320         BEEP 300,.2
330         PRINT TABXY(60,16),CHR$(137);" PULLED OUT "
340         PRINT TABXY(60,17)," SPOLL=";S
350     CASE NOT BIT(S,3) AND BIT(S,2) AND NOT BIT(S,1) AND NOT BIT(S,0)
360         Err_flg=1
370         BEEP 300,.2
380         PRINT TABXY(60,16),CHR$(137);"** ERROR! **"
390         PRINT TABXY(60,17)," SPOLL=";S
400     CASE BIT(S,3) AND BIT(S,2) AND NOT BIT(S,1) AND NOT BIT(S,0)
410         PRINT TABXY(46,19),CHR$(129);Tot_count;CHR$(128)
420         Tot_count=Tot_count+1
430     CASE BIT(S,3) AND BIT(S,2) AND BIT(S,1) AND NOT BIT(S,0)
440         BEEP 300,.2
450         PRINT TABXY(60,16),CHR$(137);"** FAILED **"
460         PRINT TABXY(60,17)," SPOLL=";S
470     CASE BIT(S,3) AND BIT(S,2) AND BIT(S,1) AND BIT(S,0)
480         BEEP 300,.2
490         PRINT TABXY(60,16),CHR$(137);"** PASSED **"
500         PRINT TABXY(60,17)," SPOLL=";S
```



AL-4706-3A COMMUNICATION LINK

```
510      CASE BIT(S,3) AND NOT BIT(S,2) AND NOT BIT(S,1) AND NOT BIT(S,
0)
520          BEEP 300,.2
530          PRINT TABXY(60,16),CHR$(137);"** LOCKED **"
540          PRINT TABXY(60,17)," SPOLL=";S
550      CASE ELSE
560          BEEP 300,.2
570          PRINT TABXY(60,16),CHR$(137);" UNRESOLVED "
580          PRINT TABXY(60,17)," SPOLL=";S
590      END SELECT
600      PRINT TABXY(1,1),CHR$(139)
610      GOTO Sub_end
620      !
630 Answer: ENTER Device;Data$
640      IF LEN(Data$)=6 AND Data$[1,1]<>"S" OR Data$[1,1]="-" THEN GOT
0 Increment
650      SELECT Data$[1,1]
660      CASE "0"
670          SELECT Data$[5,5]
680          CASE "P"                                !Answer on "X1<" query:
690              IF Data$[6,6]="-" THEN
700                  PRINT TABXY(13,7),Data$[6,9];".";Data$[10];" deg"
710              ELSE
720                  PRINT TABXY(13,7),Data$[6,8];".";Data$[9];" deg "
730              END IF
740              PRINT TABXY(13,5),Data$[4,4]          ! DFR axis number
750              Answ_x1=1
760          CASE "V"                                !Answer on "X3<" query:
770              PRINT TABXY(13,9),Data$[6,8];".";Data$[9,10];" deg/sec"
780              IF Contr_type$="4706-3-2A" THEN
790                  PRINT TABXY(39,9),Data$[16,18];".";Data$[19];" deg/sec"
800              END IF
810              Answ_x3=1
820          END SELECT
830      CASE "D"                                !Answer on "X2<" query:
840          IF Data$[6,6]="-" THEN
850              PRINT TABXY(39,7),Data$[6,9];".";Data$[10];" deg"
860          ELSE
870              PRINT TABXY(39,7),Data$[6,8];".";Data$[9];" deg "
880          END IF
890          PRINT TABXY(39,5),Data$[4,4]              ! DSFL axis number
900          Answ_x2=1
910      CASE "S"                                !Answer on "READ" query
920          Newparam=1
930          GOTO Sub_end
940      CASE ELSE
950          GOTQ System_stat                        !Answer on "X4<" query
960      END SELECT
970      GOTO Sub_end
980      !
```



AL-4706-3A COMMUNICATION LINK

```
990 System_stat: !Answ_x4=1                                "X4<" query not in use
1000 GOTO Sub_end
1010 !
1020 Increment: IF Data$(1,1)<>"-" THEN Data$=" "&Data$
1030 PRINT TABXY(X,Y);CHR$(129);Data$
1040 PRINT TABXY(46,19),Tot_count;CHR$(128)
1050 IF Tot_count-1>0 THEN PRINT TABXY(Prv_x,Prv_y),Prv_incr$
1060 Tot_count=Tot_count+1
1070 Prv_x=X
1080 Prv_y=Y
1090 Prv_incr$=Data$
1100 X=X+7
1110 IF X>42 THEN
1120     X=3
1130     Y=Y+1
1140     IF Y>19 THEN Y=13
1150 END IF
1160 !
1170 !
1180 !*****
1190 !*          END          of      SRQ_INTR-Enter Routine      *
1200 !*****
1210 !
1220 Sub_end:  ENABLE INTR Hpib
1230 SUBEND
```



4.5 Setting Up Different Motions - Loading, Verification & Monitoring

This paragraph includes programs which activate all the available motion modes according to the "Proper order of work" Flow chart (Fig. 3.3). Each of the programs consist, therefore, of 3 main parts:

- * Loading parameters
- * Reaching them back for verification
- * Running the positioner while monitoring the motion.

The use of different data entering options is also demonstrated in these programs.



AL-4706-3A COMMUNICATION LINK

4.5.1 Setting Up a Track to Target Motion



```
10      ! Setting up a Track to Target movement
20      !
30      !*****
40      !*          ***  W A R N I N G  ***          *
50      !*      Be sure to adapt the loaded parameters to your system:      *
60      !*      - axis selectin, according to the connected axes            *
70      !*      - position targets, according to the limit switches          *
80      !*      Assure that the configuration parameters are set correctly    *
90      !*****
100     !
110     !
120     !*****
130     !*      This program enters data using PPOLL method                  *
140     !*****
150     !
160     LOADSUB ALL FROM "ENTERPPOLL"
170     !
180     !
190     COM /Ieee_addr/ Device,Hpib
200     COM /In_data/ Data$(20)
210     COM /Flag/ Newparam,Answ_x1,Answ_x2,Answ_x3,Answ_x4
220     COM /Incr_dspl/ X,Y,Tot_count,Prv_incr$(15),Prv_x,Prv_y
230     !
240     Hpib=7                      !Hpib select code
250     Device=704                  !CONTROLLER IEEE-488 ADDRESS
260     !
270     RESET Hpib                  !Clear COMPUTERS Buffer
280     CLEAR Device                !Clear CONTROLLERS Buffer
290     WAIT 1                      !Wait for a sec
300     !
310     Newparam=0                  !Initiate the flags
320     Answ_x1=0
330     Answ_x2=0
340     Answ_x3=0
350     !
360     Track_lbls                  !Label the screen
370     !
380     !***** LOAD PARAMETRS *****
390     !
400     !
410     OUTPUT Device;"H"           !Turn CONTROLLER to "STAND-BY" State
420     !
430     OUTPUT Device;"L"           !Turn CONTROLLER to "LOAD" State
440     !
450     OUTPUT Device;"C43<"       !Configure the PPOLL byte
460     !
```



AL-4706-3A COMMUNICATION LINK

```
470 OUTPUT Device;"Aa1<"      !Select first rotation axis No:1
480 OUTPUT Device;"Ae3<"      !Select second rotation axis No:3
490 OUTPUT Device;"Pat05000<" !Set target of first axis :50.00 deg
500 OUTPUT Device;"Pat35180<" !Set target of first axis :351.8 deg
510 OUTPUT Device;"Daf<"      !Set direction of first axis :forward
520 OUTPUT Device;"Der<"      !Set direction of second axis :reverse
530 OUTPUT Device;"Va00876<"  !Set travel velocity of axis a:87.6%
540 OUTPUT Device;"Ve00564<"  !Set travel velocity of axis e:56.4%
550 OUTPUT Device;"MT"        !Set the motion mode:Track
560 !
570 OUTPUT Device;"H"          !Return to "STAND-BY"
580 !
590 !
600 !***** READ PARAMETERS *****
610 !
620 !
630 OUTPUT Device;"S"          !Turn CONTROLLER to "READ" State
640 !
650 OUTPUT Device;"Aa<"        !Query first rotation axis No
660 GOSUB Read_param
670 PRINT TABXY(67,4),Data$(4,4]
680 OUTPUT Device;"Ae<"        !Query second rotation axis No
690 GOSUB Read_param
700 PRINT TABXY(67,9),Data$(4,4]
710 OUTPUT Device;"Pat<"      !Query target of first axis
720 GOSUB Read_param
730 PRINT TABXY(66,5),VAL(Data$(2,1))/100;" deg"
740 OUTPUT Device;"Pet<"      !Query target of second axis
750 GOSUB Read_param
760 PRINT TABXY(66,10),VAL(Data$(2,1))/100;" deg"
770 OUTPUT Device;"Da<"        !Query direction of first axis
780 GOSUB Read_param
790 SELECT Data$(4,4]
800     CASE "<"
810         PRINT TABXY(67,6),"FRWD"
820     CASE "="
830         PRINT TABXY(67,6),"RVRS"
840     CASE "d"
850         PRINT TABXY(67,6),"AUTO"
860 END SELECT
870 OUTPUT Device;"De<"        !Query direction of second axis
880 GOSUB Read_param
890 SELECT Data$(4,4]
900     CASE "<"
910         PRINT TABXY(67,11),"FRWD"
920     CASE "="
930         PRINT TABXY(67,11),"RVRS"
940     CASE "d"
950         PRINT TABXY(67,11),"AUTO"
960 END SELECT
```



```
970  OUTPUT Device;"Va<"          !Query the travel velocity of axis a
980  GOSUB Read_param
990  PRINT TABXY(66,7),VAL(Data$(21))/10;" %"
1000 OUTPUT Device;"Ve<"          !Query the travel velocity of axis e
1010 GOSUB Read_param
1020 PRINT TABXY(66,12),VAL(Data$(21))/10;" %"
1030  !
1040 OUTPUT Device;"H"            !Return to "STAND-BY"
1050  !
1060 WAIT .010                    !Wait 10msec between 'H' & 'G'
1070  !
1080 !***** START MOTION *****
1090  !
1100  !
1110 OUTPUT Device;"G"            !Turn CONTROLLER to "RUN" State
1120  !
1130  !
1140 !***** MONITOR MOTION *****
1150  !
1160  !
1170 OUTPUT Device;"E1<"          !Select the DISPLAY axis No:1
1180  !
1190 Monitor_loop: OUTPUT Device;"X1<"
1200  !
1210  REPEAT
1220    Enter_ppoll
1230    UNTIL Answ_x1
1240    Answ_x1=0
1250    !
1260    OUTPUT Device;"X2<"
1270    !
1280    REPEAT
1290      Enter_ppoll
1300      UNTIL Answ_x2
1310      Answ_x2=0
1320      !
1330      OUTPUT Device;"X3<"
1340      !
1350      REPEAT
1360        Enter_ppoll
1370        UNTIL Answ_x3
1380        Answ_x3=0
1390        !
1400      GOTO Monitor_loop
1410  !
1420  !
```



```
1430 !***** READ PARAMETER SUBROUTINE *****
1440 !
1450 Read_param: REPEAT
1460             Enter_ppoll
1470             UNTIL Newparam
1480             Newparam=0
1490             RETURN
1500 !
1510 !
1520 !***** END *****
1530 !
1540 END
1550 !
1560 !
1570 !*****
1580 !             TRACK   SCREEN LABELING
1590 !*****
1600 !
1610 SUB Track_lbls
1620 !
1630     Green=139
1640     White=136
1650     Inverse=129
1660     Undrln=132
1670     Enhoff=128
1680 !
1690     CLEAR SCREEN
1700 !
1710     ! DRAW FRAME
1720 !
1730     GINIT
1740     VIEWPORT 0,133,23,100
1750     PEN 1
1760     FRAME
1770 !
1780     MOVE 83,88
1790     DRAW 83,7
1800 !
1810     !HEADER
1820 !
1830     CSIZE 3.75
1840     MOVE 35,93
1850     PEN 1
1860     AREA PEN 5
1870     RECTANGLE 60,4,FILL,EDGE
1880     MOVE 65,94.75
1890     LORG 5
1900     PEN 6
1910     LABEL "*** TRACK to TARGET ***"
1920 !
```



```
1930      ! CLORED LABELS
1940      !
1950      PRINT CHR$(White)
1960      PRINT TABXY(56,4),"Axis  A:"
1970      PRINT TABXY(56,5),"Target A:"
1980      PRINT TABXY(56,6),"Dirctn A:"
1990      PRINT TABXY(56,7),"Velcty A:"
2000      PRINT TABXY(56,9),"Axis  E:"
2010      PRINT TABXY(56,10),"Target E:"
2020      PRINT TABXY(56,11),"Dirctn E:"
2030      PRINT TABXY(56,12),"Velcty E:"
2040      !
2050      ! UNDERLINED LABELS
2060      !
2070      PRINT CHR$(Undrln)
2080      PRINT TABXY(3,5),"AXIS  No:"
2090      PRINT TABXY(29,5),"AXIS  No:"
2100      PRINT TABXY(3,7),"POSITION:"
2110      PRINT TABXY(29,7),"POSITION:"
2120      PRINT TABXY(3,9),"VELOCITY:"
2130      PRINT CHR$(Enhoff)
2140      !
2150      ! INVERSE VIDEO LABELS
2160      !
2170      PRINT CHR$(Green)
2180      PRINT CHR$(Inverse)
2190      PRINT TABXY(3,3),"  OPRATING AXIS  "
2200      PRINT TABXY(29,3),"  DISPLAYED AXIS  "
2210      PRINT TABXY(55,3),"  SELECTED PARAMETERS  "
2220      PRINT TABXY(55,15),"          EVENTS          "
2230      PRINT CHR$(128)
2240      PRINT CHR$(Green)
2250      !
2260      !
2270      SUBEND
2280      !
2290      !
2300      !*****
2310      !*          E N D   o f   T R A C K          *
2320      !*****
```



4.5.2 Setting Up a RASTER SCAN Motion

Note: The "RASTER" scan in the following example will be executed either in "forward" or "reverse" direction, according to the specified direction parameter: "Daf<" or "Dar<" (not included in the program).



```
10      !   Setting up a Raster Scan movement
20      !
30      !*****
40      !*               ***   W A R N I N G   ***               *
50      !*           Be sure to adapt the loaded parameters to your system:           *
60      !*           - axis selectin,according to the connected axes                 *
70      !*           - position targets,according to the limit switches              *
80      !*           Assure that the configuration parameters are set correctly        *
90      !*                                                                           *
100     !*                                                                           *
110     !*****
120     !
130     !
140     !*****
150     !*           This program activates IEEE-488 bus using SPOLL method           *
160     !*****
170     !
180     LOADSUB ALL FROM "ENTERSPOLL"
190     !
200     COM /Ieee_addr/ Device,Hpib
210     COM /In_data/ Data$[20]
220     COM /Flag/ Newparam,Answ_x1,Answ_x2,Answ_x3,Answ_x4
230     COM /Incr_dspl/ X,Y,Tot_count,Prv_incr$[15],Prv_x,Prv_y
240     !
250     Hpib=7                      !Hpib select code
260     Device=704                  !CONTROLLER IEEE-488 ADDRESS
270     !
280     RESET Hpib                  !CLEAR Computers BUFFER
290     CLEAR Device                !CLEAR Controllers BUFFER
300     WAIT 1                      !Wait for a sec
310     !
320     X=3                         !Increments display initialization
330     Y=13
340     Tot_count=1
350     !
360     Newparam=0                  !Initiate the flagS
370     Answ_x1=0
380     Answ_x2=0
390     Answ_x3=0
400     !
410     Raster_lbls                 !Label the screen
420     !
```



```
430 !***** LOAD PARAMETERS *****
440 !
450 !
460 OUTPUT Device;"H"           !Turn CONTROLLER to "STAND-BY" State
470 !
480 OUTPUT Device;"L"           !Turn CONTROLLER to "LOAD" State
490 !
500 OUTPUT Device;"C43<"       !Configure the PPOLL byte
510 !
520 OUTPUT Device;"Aa1<"       !Select scan axis No:1
530 OUTPUT Device;"Ae4<"       !Select step axis No:4
540 OUTPUT Device;"Pas01000<"  !Set scan starting point :10.00 deg
550 OUTPUT Device;"Pab05000<"  !Set scan ending point :50.00 deg
560 OUTPUT Device;"Pes02500<"  !Set step starting point :25.00 deg
570 OUTPUT Device;"Peb04000<"  !Set step ending point :40.00 deg
580 OUTPUT Device;"Va00765<"   !Set scan velocity :76.5%
590 OUTPUT Device;"Ve00234<"   !Set step velocity :23.4%
600 OUTPUT Device;"J00500<"    !Set step value :5 deg
610 OUTPUT Device;"I02000<"    !Set increment gap :2 deg
620 OUTPUT Device;"MR"         !Set the motion mode :Raster A
630 !
640 OUTPUT Device;"H"           !Return to "STAND-BY"
650 !
660 !
670 !***** READ PARAMETERS *****
680 !
690 !
700 OUTPUT Device;"S"           !Turn CONTROLLER to "READ" State
710 !
720 OUTPUT Device;"Aa<"        !Query scan axis No
730 GOSUB Read_param
740 PRINT TABXY(67,4),Data$(4,4]
750 OUTPUT Device;"Ae<"        !Query step axis No
760 GOSUB Read_param
770 PRINT TABXY(67,5),Data$(4,4]
780 OUTPUT Device;"Pas<"       !Query scan starting point
790 GOSUB Read_param
800 PRINT TABXY(66,6),VAL(Data$(2))/100;" deg"
810 OUTPUT Device;"Pab<"       !Query scan ending point
820 GOSUB Read_param
830 PRINT TABXY(66,7),VAL(Data$(2))/100;" deg"
840 OUTPUT Device;"Pes<"       !Query step starting point
850 GOSUB Read_param
860 PRINT TABXY(66,8),VAL(Data$(2))/100;" deg"
870 OUTPUT Device;"Peb<"       !Query step ending point
880 GOSUB Read_param
```



```
890 PRINT TABXY(66,9),VAL(Data$(2))/100;" deg"
900 OUTPUT Device;"Va<" !Query scan velocity
910 GOSUB Read_param
920 PRINT TABXY(66,10),VAL(Data$(2))/10;" %"
930 OUTPUT Device;"Ve<" !Query step velocity
940 GOSUB Read_param
950 PRINT TABXY(66,11),VAL(Data$(2))/10;" %"
960 OUTPUT Device;"J<" !Query the step value
970 GOSUB Read_param
980 PRINT TABXY(66,12),VAL(Data$(2))/100;" deg"
990 OUTPUT Device;"I<" !Query the increment gap
1000 GOSUB Read_param
1010 PRINT TABXY(66,13),VAL(Data$(2))/1000;" deg"
1020 !
1030 OUTPUT Device;"H" !Return to "STAND-BY"
1040 !
1050 WAIT .010 !Wait 10msec between 'H' & 'G'
1060 !
1070 !***** START MOTION *****
1080 !
1090 !
1100 OUTPUT Device;"G" !Turn CONTROLLER to "RUN" State
1110 !
1120 !
1130 !***** MONITOR MOTION *****
1140 !
1150 !
1160 OUTPUT Device;"E1<" !Select the DISPLAY axis No:1
1170 !
1180 Monitor_loop: OUTPUT Device;"X1<"
1190 !
1200 FOR I=1 TO 100
1210 Enter_spoll
1220 IF Answ_x1=1 THEN
1230 Answ_x1=0
1240 GOTO Next_x2
1250 END IF
1260 NEXT I
1270 !
1280 Next_x2: OUTPUT Device;"X2<"
1290 !
1300 FOR I=1 TO 100
1310 Enter_spoll
1320 IF Answ_x2=1 THEN
1330 Answ_x2=0
1340 GOTO Next_x3
1350 END IF
1360 NEXT I
1370 !
```



```
1380 Next_x3:      OUTPUT Device;"X3<"
1390              !
1400              FOR I=1 TO 100
1410                  Enter_poll
1420                  IF Answ_x3=1 THEN
1430                      Answ_x3=0
1440                      GOTO Monitor_loop
1450                  END IF
1460              NEXT I
1470              !
1480              GOTO Monitor_loop
1490              !
1500              !
1510              !***** READ PARAMETER SUBROUTINE *****
1520              !
1530 Read_param:    REPEAT
1540                  Enter_poll
1550                  UNTIL Newparam
1560                  Newparam=0
1570                  RETURN
1580              !
1590              !
1600              !***** END *****
1610              !
1620              END
1630              !
1640              !*****
1650              !          RASTER SCREEN LABELS
1660              !*****
1670              !
1680              SUB Raster_lbls
1690              !
1700              Green=139
1710              White=136
1720              Inverse=129
1730              Undrln=132
1740              Enhoff=128
1750              !
1760              CLEAR SCREEN
1770              !
1780              ! DRAW FRAME
1790              !
1800              GINIT
1810              VIEWPORT 0,133,23,100
1820              PEN 1
1830              FRAME
```



```
1840      !
1850      MOVE 83,98
1860      DRAW 83,7
1870      !
1880      !HEADER
1890      !
1900      CSIZE 3.75
1910      MOVE 35,93
1920      PEN 1
1930      AREA PEN 5
1940      LORG 5
1950      RECTANGLE 60,4,FILL,EDGE
1960      MOVE 65,94.75
1970      PEN 6
1980      LABEL "*** RASTER SCAN A ***"
1990      !
2000      ! COLORED LABELS
2010      !
2020      PRINT CHR$(White)
2030      PRINT TABXY(56,4),"Scan axis:"
2040      PRINT TABXY(56,5),"Step axis:"
2050      PRINT TABXY(56,6),"Scan strt:"
2060      PRINT TABXY(56,7),"Scan end :"
2070      PRINT TABXY(56,8),"Step strt:"
2080      PRINT TABXY(56,9),"Step end :"
2090      PRINT TABXY(56,10),"Scan vel :"
2100      PRINT TABXY(56,11),"Step vel :"
2110      PRINT TABXY(56,12),"Stepvalue:"
2120      PRINT TABXY(56,13),"Increment:"
2130      PRINT TABXY(46,18),"Total:"
2140      !
2150      !UNDERLINED LABELS
2160      !
2170      PRINT CHR$(Undrln)
2180      PRINT TABXY(3,5),"AXIS No:"
2190      PRINT TABXY(29,5),"AXIS No:"
2200      PRINT TABXY(3,7),"POSITION:"
2210      PRINT TABXY(29,7),"POSITION:"
2220      PRINT TABXY(3,9),"VELOCITY:"
2230      PRINT CHR$(Enhoff)
```



AL-4706-3A COMMUNICATION LINK

```
2240      !
2250      ! INVERSE VIDEO LABELS
2260      !
2270      PRINT CHR$(Green)
2280      PRINT CHR$(Inverse)
2290      PRINT TABXY(3,3)," OPERATING AXIS "
2300      PRINT TABXY(29,3)," DISPLAYED AXIS "
2310      PRINT TABXY(55,3)," SELECTED PARAMETERS "
2320      PRINT TABXY(55,14),"          EVENTS          "
2330      PRINT TABXY(3,11),"          ***** I N C R E M E N T S *****
"
2340      PRINT CHR$(128)
2350      PRINT CHR$(Green)
2360      !
2370      !
2380      SUBEND
2390      !
2400      !
2410      !*****
2420      !*          E N D   o f   R A S T E R          *
2430      !*****
```



4.5.3 Setting Up a SECTOR SCAN Motion

Note: The "SECTOR" scan in the following example will be executed either in "forward" or "reverse" direction, according to the specified direction parameter: "Daf<" or "Dar<" (not included in the program).



```
10      !   Setting up a Sector Scan movement
20      !
30      !*****
40      !*           ***   W A R N I N G   ***           *
50      !*           Be sure to adapt the loaded parameters to your system:           *
60      !*           - axis selectin,according to the connected axes                   *
70      !*           - position targets,according to the limit switches                 *
80      !*           Assure that the configuration parameters are set correctly         *
90      !*                                                                           *
100     !*                                                                           *
110     !*****
120     !
130     !
140     !*****
150     !*           This program activates IEEE-488 bus using SRQ_INTR method           *
160     !*****
170     !
180     LOADSUB ALL FROM "ENTERINTR"
190     !
200     COM /Ieee_addr/ Device,Hpib
210     COM /In_data/ Data$[20]
220     COM /Flag/ Newparam,Answ_x1,Answ_x2,Answ_x3,Answ_x4
230     COM /Incr_dspl/ X,Y,Tot_count,Prv_incr$[15],Prv_x,Prv_y
240     !
250     Hpib=7                                !Hpib select code
260     Device=704                            !CONTROLLER IEEE-488 ADDRESS
270     !
280     RESET Hpib                            !CLEAR Computers BUFFER
290     CLEAR Device                          !CLEAR Controllers BUFFER
300     WAIT 1                               !Wait for a sec
310     !
320     !***** HPIB INTERRUPT INITIALIZATION *****
330     !
340     Mask=2
350     Priority=5
360     !
370     ON INTR Hpib,Priority CALL Enter_intr
380     ENABLE INTR Hpib;Mask
390     !
400     !*****
410     !
420     X=3                                    !Increments display initialization
430     Y=13
440     Tot_count=1
450     !
```



AL-4706-3A COMMUNICATION LINK

```
460 Newparam=0                !Initiate the flagS
470 Answ_x1=0
480 Answ_x2=0
490 Answ_x3=0
500 !
510 Sector_lbls                !Label the screen
520 !
530 !***** LOAD PARAMETERS *****
540 !
550 !
560 OUTPUT Device;"H"          !Turn CONTROLLER to "STAND-BY" State
570 !
580 OUTPUT Device;"L"          !Turn CONTROLLER to "LOAD" State
590 !
600 OUTPUT Device;"C43<"      !Configure the PPOLL byte
610 !
620 OUTPUT Device;"Aa1<"      !Select scan axis No:1
630 OUTPUT Device;"Ae0<"      !Select step axis No:0
640 OUTPUT Device;"Pas34000<" !Set scan starting point :340.00 deg
650 OUTPUT Device;"Pab02000<" !Set scan ending point :20.00 deg
660 OUTPUT Device;"Va00500<"  !Set scan velocity :50%
670 OUTPUT Device;"Ve00999<"  !Set start velocity :99.9%
680 OUTPUT Device;"J00005<"   !Set number of scans :5
690 OUTPUT Device;"I02000<"   !Set increment gap :2 deg
700 OUTPUT Device;"MN"        !Set the motion mode :Raster B
710 !                          (might as well be RASTER A)
720 !
730 OUTPUT Device;"H"          !Return to "STAND-BY"
740 !
750 !
760 !***** READ PARAMETERS *****
770 !
780 !
790 OUTPUT Device;"S"          !Turn CONTROLLER to "READ" State
800 !
810 OUTPUT Device;"Aa<"        !Query scan axis No
820 GOSUB Read_param
830 PRINT TABXY(67,5),Data$(4,4)
840 OUTPUT Device;"Pas<"      !Query scan starting point
850 GOSUB Read_param
860 PRINT TABXY(66,6),VAL(Data$(2))/100;" deg"
870 OUTPUT Device;"Pab<"      !Query scan ending point
880 GOSUB Read_param
890 PRINT TABXY(66,7),VAL(Data$(2))/100;" deg"
900 OUTPUT Device;"Va<"        !Query scan velocity
910 GOSUB Read_param
```



```
920 PRINT TABXY(66,8),VAL(Data$(2))/10;" %"
930 OUTPUT Device;"V<" !Query start velocity
940 GOSUB Read_param
950 PRINT TABXY(66,9),VAL(Data$(2))/10;" %"
960 OUTPUT Device;"J<" !Query the number of scans
970 GOSUB Read_param
980 PRINT TABXY(66,10),VAL(Data$(2));" "
990 OUTPUT Device;"I<" !Query the increment gap
1000 GOSUB Read_param
1010 PRINT TABXY(66,11),VAL(Data$(2))/1000;" deg"
1020 !
1030 OUTPUT Device;"H" !Return to "STAND-BY"
1040 !
1050 WAIT .010 !Wait 10msec between 'H' & 'G'
1060 !
1070 !***** START MOTION *****
1080 !
1090 !
1100 OUTPUT Device;"G" !Turn CONTROLLER to "RUN" State
1110 !
1120 !
1130 !***** MONITOR MOTION *****
1140 !
1150 !
1160 OUTPUT Device;"E1<" !Select the DISPLAY axis No:1
1170 !
1180 Monitor_loop: OUTPUT Device;"X1<"
1190 !
1200 FOR I=1 TO 100
1210 WAIT .01
1220 IF Answ_x1=1 THEN
1230 Answ_x1=0
1240 GOTO Next_x2
1250 END IF
1260 NEXT I
1270 !
1280 Next_x2: OUTPUT Device;"X2<"
1290 !
1300 FOR I=1 TO 100
1310 WAIT .01
1320 IF Answ_x2=1 THEN
1330 Answ_x2=0
1340 GOTO Next_x3
1350 END IF
1360 NEXT I
1370 !
```



```
1380 Next_x3:      OUTPUT Device;"X3<"
1390              !
1400              FOR I=1 TO 100
1410                  WAIT .01
1420                  IF Answ_x3=1 THEN
1430                      Answ_x3=0
1440                      GOTO Monitor_loop
1450                  END IF
1460              NEXT I
1470              !
1480              GOTO Monitor_loop
1490          !
1500          !
1510          !***** READ PARAMETER SUBROUTINE *****!
1520          !
1530          Read_param:  REPEAT
1540                      UNTIL Newparam
1550                      Newparam=0
1560                      RETURN
1570          !
1580          !
1590          !***** END *****!
1600          !
1610          END
1620          !
1630          !***** SECTOR SCREEN LABELS *****!
1640          !
1650          !*****
1660          !
1670              SUB Sector_lbls
1680              !
1690              Green=139
1700              White=136
1710              Inverse=129
1720              Undrln=132
1730              Enhoff=128
1740              !
1750              CLEAR SCREEN
1760              !
1770              ! DRAW FRAME
1780              !
1790              GINIT
1800              VIEWPORT 0,133,23,100
1810              PEN 1
1820              FRAME
1830              !
1840              MOVE 83,88
1850              DRAW 83,7
```



```
1860      !
1870      !HEADER
1880      !
1890      CSIZE 3.75
1900      MOVE 35,93
1910      PEN 1
1920      AREA PEN 5
1930      LORG 5
1940      RECTANGLE 60,4,FILL,EDGE
1950      MOVE 65,94.75
1960      PEN 6
1970      LABEL "*** SECTOR SCAN B ***"
1980      !
1990      ! COLORED LABELS
2000      !
2010      PRINT CHR$(White)
2020      PRINT TABXY(56,5),"Scan axis:"
2030      PRINT TABXY(56,6),"Scan strt:"
2040      PRINT TABXY(56,7),"Scan end :"
2050      PRINT TABXY(56,8),"Scan vel  :"
2060      PRINT TABXY(56,9),"Start vel:"
2070      PRINT TABXY(56,10),"Scans No.:"
2080      PRINT TABXY(56,11),"Increment:"
2090      PRINT TABXY(46,18),"Total:"
2100      !
2110      !UNDERLINED LABELS
2120      !
2130      PRINT CHR$(Undrln)
2140      PRINT TABXY(3,5),"AXIS  No:"
2150      PRINT TABXY(29,5),"AXIS  No:"
2160      PRINT TABXY(3,7),"POSITION:"
2170      PRINT TABXY(29,7),"POSITION:"
2180      PRINT TABXY(3,9),"VELOCITY:"
2190      PRINT CHR$(Enhoff)
2200      !
2210      ! INVERSE VIDEO LABELS
2220      !
2230      PRINT CHR$(Green)
2240      PRINT CHR$(Inverse)
2250      PRINT TABXY(3,3)," OPERATING AXIS "
2260      PRINT TABXY(29,3)," DISPLAYED AXIS "
2270      PRINT TABXY(55,3)," SELECTED PARAMETERS "
2280      PRINT TABXY(55,14),"          EVENTS          "
2290      PRINT TABXY(3,11),"          ***** I N C R E M E N T S *****"
"
```



AL-4706-3A COMMUNICATION LINK

```
2300      PRINT CHR$(128)
2310      PRINT CHR$(Green)
2320      !
2330      !
2340      SUBEND
2350      !
2360      !
2370      !*****
2380      !*                E N D   o f   S E C T O R                *
2390      !*****
```



4.5.4 Setting Up a SLEW Motion



```
10      !   Setting up a Slew movement
11      !
12      !*****
13      !*                                     ***   W A R N I N G   ***
14      !*      Be sure to adapt the loaded parameters to your system:
15      !*      - axis selectin,according to the connected axes
16      !*      - position targets,according to the limit switches
17      !*      Assure that the configuration parameters are set correctly
18      !*****
19      !
20      !
21      !*****
22      !*      This program activates IEEE-488 bus using SPOLL method
23      !*****
24      !
25      !
26      LOADSUB ALL FROM "ENTERSPOLL"
27      !
28      !
29      COM /Ieee_addr/ Device,Hpib
30      COM /In_data/ Data$[20]
31      COM /Flag/ Newparam,Answ_x1,Answ_x2,Answ_x3,Answ_x4
32      COM /Incr_dspl/ X,Y,Tot_count,Prv_incr$[15],Prv_x,Prv_y
33      !
34      Hpib=7                      !Hpib select code
35      Device=704                  !CONTROLLER IEEE-488 ADDRESS
36      !
37      RESET Hpib                  !CLEAR Computers BUFFER
38      CLEAR Device                !CLEAR Controllers BUFFER
39      WAIT 1                      !Wait for a sec
40      !
41      Newparam=0                  !Initiate the flags
42      Answ_x1=0
43      Answ_x2=0
44      Answ_x3=0
45      Ff=0
46      !
47      !***** SOFT-KEY LABELING *****
48      !
49      ON KEY 1 LABEL " DIRCTN  TOGGLE " GOSUB Slew_run
50      ON KEY 8 LABEL "  STOP  " GOSUB Stop
51      FOR T=2 TO 7
52      ON KEY T LABEL "          " GOSUB Nothing
53      NEXT T
54      CONTROL 2,2;1
55      !
```



```
470 !*****
480 !
490 Slew_lbls !Label the screen
500 !
510 !***** LOAD PARAMETERS *****
520 !
530 !
540 OUTPUT Device;"H" !Turn CONTROLLER to "STAND-BY" State
550 !
560 OUTPUT Device;"L" !Turn CONTROLLER to "LOAD" State
570 !
580 OUTPUT Device;"C43<" !Configure the PPOLL byte
590 !
600 OUTPUT Device;"Az3<" !Select slew axis No:3
610 OUTPUT Device;"Vz00654<" !Set slew velocity :65.4%
620 OUTPUT Device;"MZ" !Set the motion mode :Slew
630 !
640 OUTPUT Device;"H" !Return to "STAND-BY"
650 !
660 !
670 !***** READ PARAMETERS *****
680 !
690 !
700 OUTPUT Device;"S" !Turn CONTROLLER to "READ" State
710 !
720 OUTPUT Device;"Az<" !Query slew axis No
730 GOSUB Read_param
740 PRINT TABXY(67,6),Data$(4,4]
750 OUTPUT Device;"Vz<" !Query slew velocity
760 GOSUB Read_param
770 PRINT TABXY(66,7),VAL(Data$(2))/10;" %"
780 !
790 OUTPUT Device;"H" !Return to "STAND-BY"
800 !
810 !
820 !***** STAND BY TO SLEW *****
830 !
840 !
850 OUTPUT Device;"G" !Turn CONTROLLER to "RUN" State
860 !
870 !
```



AL-4706-3A COMMUNICATION LINK

```
880 ***** MONITOR MOTION *****
890 !
900 !
910 OUTPUT Device;"E1<"          !Select the DISPLAY axis No:1
920 !
930 Monitor_loop:  OUTPUT Device;"X1<"
940 !
950 REPEAT
960     Enter_spoll
970 UNTIL Answ_x1
980 Answ_x1=0
990 !
1000 OUTPUT Device;"X2<"
1010 !
1020 REPEAT
1030     Enter_spoll
1040 UNTIL Answ_x2
1050 Answ_x2=0
1060 !
1070 OUTPUT Device;"X3<"
1080 !
1090 REPEAT
1100     Enter_spoll
1110 UNTIL Answ_x3
1120 Answ_x3=0
1130 !
1140 GOTO Monitor_loop
1150 !
1160 !
1170 ***** READ PARAMETER SUBROUTINE *****
1180 !
1190 Read_param:  REPEAT
1200     Enter_spoll
1210 UNTIL Newparam
1220 Newparam=0
1230 RETURN
1240 !
```



```
1250 !***** TOGGLE SLEW DIRECTION *****
1260 !
1270 Slew_run: Ff=NOT Ff
1280 IF Ff THEN
1290     BEEP 500,.2
1300     BEEP 1000,.2
1310     OUTPUT Device;"k"           ! Forward
1320 ELSE
1330     BEEP 1000,.2
1340     BEEP 500,.2
1350     OUTPUT Device;"l"           ! Reverse
1360 END IF
1370 RETURN
1380 !
1390 !***** STOP *****
1400 !
1410 Stop: OUTPUT Device;"H"
1420 RETURN
1430 !
1440 !***** NOTHING *****
1450 !
1460 Nothing: RETURN
1470 !
1480 !***** END *****
1490 !
1500 END
1510 !
1520 !*****
1530 !           SLEW SCREEN LABELS
1540 !*****
1550 !
1560 SUB Slew_lbls
1570 Green=139
1580 Red=137
1590 White=136
1600 Inverse=129
1610 Undrln=132
1620 Enhoff=128
1630 !
1640 ! DRAW FRAME
1650 !
1660 GINIT
1670 GRAPHICS ON
1680 VIEWPORT 0,133,23,100
1690 CLEAR SCREEN
1700 PEN 1
1710 FRAME
```



AL-4706-3A COMMUNICATION LINK

```
1720      !
1730      MOVE 83,86
1740      DRAW 83,5
1750      !
1760      ! COLORED LABELS
1770      !
1780      CSIZE 3.75
1790      MOVE 35,93
1800      PEN 1
1810      AREA PEN 5
1820      LONG 5
1830      RECTANGLE 60,4,FILL,EDGE
1840      MOVE 65,94.75
1850      PEN 6
1860      LABEL " ***** S L E W ***** "
1870      PRINT CHR$(White)
1880      PRINT TABXY(56,6),"Slew axis:"
1890      PRINT TABXY(56,7),"Velocity:"
1900      !
1910      ! UNDERLINED LABELS
1920      !
1930      PRINT CHR$(Undrln)
1940      PRINT TABXY(3,5),"AXIS No:"
1950      PRINT TABXY(29,5),"AXIS No:"
1960      PRINT TABXY(3,7),"POSITION:"
1970      PRINT TABXY(29,7),"POSITION:"
1980      PRINT TABXY(3,9),"VELOCITY:"
1990      PRINT CHR$(Enhoff)
2000      !
2010      ! INVERSE VIDEO LABELS
2020      !
2030      PRINT CHR$(Green)
2040      PRINT CHR$(Inverse)
2050      PRINT TABXY(3,3)," OPERATING AXIS "
2060      PRINT TABXY(29,3)," DISPLAYED AXIS "
2070      PRINT TABXY(55,3)," SELECTED PARAMETERS "
2080      PRINT TABXY(55,14),"          EVENTS          "
2090      PRINT CHR$(128)
2100      PRINT CHR$(Green)
2110      !
2120      ! SPECIAL MESSAGE
2130      !
2140      PRINT CHR$(White)
2150      PRINT TABXY(15,14),"Toggle SLEW direction"
2160      PRINT TABXY(15,15),"          with ";CHR$(Red);CHR$(Inverse);" f1 ";CHR$(
(128);CHR$(White);" key"
2170      PRINT TABXY(15,18),"To STOP press ";CHR$(Red);CHR$(Inverse);" f8 "
;CHR$(128);CHR$(White);" key"
2180      PRINT CHR$(Green)
2190      !
2200 SUBEND !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
```



4.6 Simple Applications

4.6.1 Reading GPIB Increments Using PPOLL

Note: The "RASTER" scan in the following example will be executed either in "forward" or "reverse" direction, according to the specified direction parameter: "Daf<" or "Dar<" (not included in the program).



AL-4706-3A COMMUNICATION LINK

```
10 *****
20 !*      This program executes one SECTOR while reading GPIB increments  *
30 !*      into a BUFFER. The BUFFER is printed after the SECTOR is done  *
40 !*
50 !*      The increments are read using PPOLL method                      *
60 *****
70 !
80 COM /Count/ I,A$(1000)[6]
90 I=0
100 Device=704
110 !
120 Start_pnt$="00000"           ! Sector starting point: 0 deg
130 End_pnt$="05000"           ! Sector ending point: 50 deg
140 No_of_scns$="00001"         ! No of Scans: 1
150 Incr_gap$="00100"           ! Gap between increments: 0.1 deg
160 Velocity$="00100"           ! Scan speed: 10% of max
170 !
180 CLEAR SCREEN
190 CLEAR Device
200 WAIT .5
210 !
220 OUTPUT Device;"HLAa1<Ae0<Pas";Start_pnt$;"<Pab";End_pnt$;"<J";No_of_scns$;
    "<I";Incr_gap$;"<Va";Velocity$;"<MRH"
230 !
240 WAIT .01
250 OUTPUT Device;"G"
260 !
270 Loop: !
280     P=PPOLL(7)
290     IF BIT(P,3) THEN GOTO Enter_incr
300     IF BIT(P,4) THEN GOTO Locked
310     GOTO Loop
320 Locked: S=SPOLL(Device)
330     IF S=88 THEN GOTO Print_buff
340     GOTO Loop
350 Enter_incr: ENTER Device;Incr$
360     A$(I)=Incr$
370     I=I+1
380     GOTO Loop
390 !
400 !
410 Print_buff: !
420     Incr=VAL(Incr_gap$)/1000
430     FOR I=0 TO (VAL(End_pnt$)-VAL(Start_pnt$))/Incr/100
440         PRINT I,"Increment=";A$(I)
450     NEXT I
460     PRINT
470     PRINT "END OF BUFFER"
480     !
490     END
```



4.6.2 Reading GPIB Increments Using SRQ INTERRUPT

Note: The "RASTER" scan in the following example will be executed either in "forward" or "reverse" direction, according to the specified direction parameter: "Daf<" or "Dar<" (not included in the program).



AL-4706-3A COMMUNICATION LINK

```
10 *****
20 !*      This program executes one SECTOR while reading GPIB increments      *
30 !*      into a BUFFER. The BUFFER is printed after the SECTOR is done      *
40 !*
50 !*      The increments are read using SRQ-INTERRUPT                        *
60 *****
70 !
80 COM /Count/ I,A$(1000)[6],Device,Print_flg
90 I=0
100 Print_flg=0
110 Device=704
120 !
130 Start_pnt$="00000"              ! Sector starting point: 0 deg
140 End_pnt$="05000"                ! Sector ending point: 50 deg
150 No_of_scns$="00001"             ! No of Scans: 1
160 Incr_gap$="00100"               ! Gap between increments: 0.1 deg
170 Velocity$="00100"               ! Scan speed: 10% of max
180 !
190 CLEAR SCREEN
200 CLEAR Device
210 WAIT .5
220 !
230 OUTPUT Device;"HLAa1<Ae0<Pas";Start_pnt$;"<Pab";End_pnt$;"<J";No_of_scns$;
"<I";Incr_gap$;"<Va";Velocity$;"<MRH"
240 !
250 WAIT .01
260 OUTPUT Device;"G"
270 !
280 ON INTR 7 CALL Read_incr
290 ENABLE INTR 7;2
300 !
310 !
320 Do_nothing: IF Print_flg=0 THEN GOTO Do_nothing
330 !
340 Incr=VAL(Incr_gap$)/1000
350 FOR I=0 TO (VAL(End_pnt$)-VAL(Start_pnt$))/Incr/100
360     PRINT I,"Increment=";A$(I)
370 NEXT I
380 PRINT
390 PRINT "END OF BUFFER"
400 !
410 END
420 !
430 !
```



```
440 Read_incr:  SUB Read_incr
450              COM /Count/ I,A$(*),Device,Print_flg
460              S=SFOLL(Device)
470              IF S=98 THEN
480                  Print_flg=1
490                  SUBEXIT
500              END IF
510              IF BIT(S,7) THEN
520                  ENTER Device;A$(I)
530                  I=I+1
540              END IF
550 Sub_end:     ENABLE INTR 7
560              SUBEND
```



4.6.3 Stepping Positioner Using 'TRACK' Mode

In this example, the positioner is stepped using the 'TRACK' mode: in each step a new target is output by the computer, which then waits for a 'LOCK' message (SPOLL = 88) from the CONTROLLER. The CONTROLLER outputs the 'LOCK' message according to the predefined lock window and internal measurement algorithm. After each 'LOCK' message, the CONTROLLER energizes the "Shut-Down" relay preventing the positioner from creeping.

In comparison with the alternative way of stepping (see para. 4.6.4), the advantage in this method is the simplicity of programming.



AL-4706-3A COMMUNICATION LINK

```
10  !*****
20  !*   This program steps the positioner in small intervals using the   *
30  !*   'TRACK' mode                                                     *
40  !*****
50  !
60      CLEAR SCREEN
70      Device=704
80      Start_pnt$="00000"
90      !
100     CLEAR Device
110     WAIT .5
120     !
130     !*****
140     !   Move to starting point with regular parameter setting
150     !
160     !
170     OUTPUT Device;"HLMTPat";Start_pnt$;"<Va00500<Dad<Aa1<Ae0<A1<c00003<
g00010<W00005<H"
180     WAIT .01
190     OUTPUT Device;"G"
200     !
210 Wait_for_strt:
220     S=SPOLL(Device)
230     IF NOT BIT(S,6) THEN GOTO Wait_for_strt
240     IF S<>88 THEN GOTO Wait_for_strt
250     BEEP
260     PRINT "Arrived to Starting point !"
270     !
280     !*****
290     !   Execute small steps using 'TRACK' mode:
300     !   NOTE ! -
310     !   - Gain & Accl factors should be adjusted for best performance:
320     !   quickest possible steps with smallest possible under/overshoots
330     !
340     !   GAIN & ACCL factors are VERY MUCH 'SYSTEM DEPENDENT' !!!
350     !
360     Gain$="00010"
370     Accl$="00020"
380     Lck_wndw$="00001"
390     Step=.1
400     !
410     OUTPUT Device;"HLMTVa00999<Dad<Aa1<Ae0<A1<c";Gain$;"<g";Accl$;"<W";
Lck_wndw$;"<"
420     !
430     Next_pos=VAL(Start_pnt$)/100+Step
440     Lck_wndw=VAL(Lck_wndw$)/100
450     !
```



AL-4706-3A COMMUNICATION LINK

```
460 Next_pnt:Start=TIMEDATE
470     Next_pos$=VAL$(100*Next_pos)
480     Next_pos$=Next_pos$[1,5]
490     Next_pos$=RPT$("0",5-LEN(Next_pos$))&Next_pos$
500     OUTPUT Device;"HLPat";Next_pos$;"<H"
510     WAIT .01
520     OUTPUT Device;"G"
530     !
540 Wait_for_lock: !
550     S=SPOLL(Device)
560     IF NOT BIT(S,6) THEN GOTO Wait_for_lock
570     IF S<>88 THEN GOTO Wait_for_lock
580     End=TIMEDATE
590     BEEP
600     PRINT "Locked within ";Lck_wndw;" of ";Next_pos;"deg      TIME=";PRG
UND(End-Start,-2)
610     !
620     !*****
630     !      This 'WAIT' represents a customer measurement routine
640     !      WAIT 1
650     !*****
660     !
670     Next_pos=Next_pos+Step
680     GOTO Next_pnt
690     !
700     !
710     END
```



4.6.4 Stepping Positioner Using 'DIRECT' Mode

In this example, the positioner is stepped using the 'DIRECT' mode: in each step a new target is output by the computer, which then has to continuously monitor the current position and velocity in order to decide whether the positioner has reached the target. The CONTROLLER on its behalf will neither output a 'LOCK' message nor energize the "Shut-Down" relay. The creeping of the positioner will not occur as the closed control loop is never broken.

In comparison with the alternative way of stepping (see para. 4.6.3), the advantages here are:

- quicker steps
- "Shut-Down" relay is not used so extensively.



AL-4706-3A COMMUNICATION LINK

```
10 *****
20 !*   This program steps the positioner in small intervals using the   *
30 !*   'DIRECT' mode                                                    *
40 *****
50 !
60     CLEAR SCREEN
70     Device=704
80     CLEAR Device
90     WAIT .5
100    Start_pnt=0
110    Start_pnt$="00000"
120    Update_rate=.01
130    Lck_wndw=.01
140    Step=.1
150    !
160    *****
170    ! Move to the Starting point using 'TRACK' mode with regular
180    !     gain, acceleration & lock window setting:
190    !
200    OUTPUT Device;"HLMTPat";Start_pnt$;"<Va00500<Aa1<Ae0<Dad<A1<W00003<
c00003<g00010<H"
210    WAIT .02
220    OUTPUT Device;"G"
230    !
240    Wait_for_lock: !
250        S=SPOLL(Device)
260        IF NOT BIT(S,6) THEN GOTO Wait_for_lock
270        IF S<>88 THEN GOTO Wait_for_lock
280        PRINT "STARTING POINT !"
290        BEEP
300        !
310        *****
320        !   Execute small steps using 'DIRECT' mode:
330        !       NOTE ! -
340        !       - Gain & Accl factors should be adjusted for best performance:
350        !       quickest possible steps with smallest possible under/overshoots
360        !
370        !       GAIN & ACCL factors are VERY MUCH 'SYSTEM DEPENDENT' !!!
380        !
390        Gain$="00006"
400        Accl$="00010"
410        !
420        OUTPUT Device;"HLMUVa00999<Aa1<A1<c";Gain$;"<g";Accl$;"<"
430        !
```



AL-4706-3A COMMUNICATION LINK

```
440      Next_pos=Start_pnt
450      !
460 Next_pnt:Next_pos=Next_pos+Step
470      Next_pos$=VAL$(100*Next_pos)
480      Next_pos$=Next_pos$(1,5)
490      Next_pos$=RPT$("0",5-LEN(Next_pos$))&Next_pos$
500      Start=TIMEDATE
510 Same_pnt:OUTPUT Device;"LPat";Next_pos$;"<G"
520      WAIT Update_rate
530      !
540      ! Note that here, instead of waiting for the 'LOCK' message,
550      ! the computer measures continuously the position & velocity,
560      ! so it can decide by itself whether in place or not yet !
570      !
580      OUTPUT Device;"X1<"
590 Wait_for_pos:!
600      S=SPOLL(Device)
610      IF NOT BIT(S,6) OR NOT BIT(S,7) THEN GOTO Wait_for_pos
620      ENTER Device;Data$
630      Act_pos=VAL(Data$[6,11])/1000
640      !
650      OUTPUT Device;"X3<"
660 Wait_for_vel:!
670      S=SPOLL(Device)
680      IF NOT BIT(S,6) OR NOT BIT(S,7) THEN GOTO Wait_for_vel
690      ENTER Device;Data$
700      Velocity=VAL(Data$[6,10])/10
710      Delta=ABS(ROUND((Act_pos-Next_pos),-2))
720      IF Delta<=Lck_wndw AND Velocity<.5 THEN
730          End=TIMEDATE
740          BEEP
750          PRINT "Locked on Positon=";Act_pos,"TIME=";ROUND(End-Start,-2)
760          !
770          !*****
780          ! This 'WAIT' represents a customer measurement routine
790          ! WAIT 1
800          !*****
810          !
820          GOTO Next_pnt
830      END IF
840      GOTO Same_pnt
850      !
860      !
870      END
```



4.6.5 Counting Increments by SRQ using SPOLL

Note: The "RASTER" scan in the following example will be executed either in "forward" or "reverse" direction, according to the specified direction parameter: "Daf<" or "Dar<" (not included in the program).



AL-4706-3A COMMUNICATION LINK

```
10 *****
20 !* This program executes one SECTOR while counting GPIB increments. *
30 !* The CONTROLLER is instructed to output just the SRQ byte for each *
40 !* increment: the position update is suppressed by the 'Fe<' command. *
50 !* *
60 !* The increments are counted using SPOLL method *
70 *****
80 !
90 Incr_count=0
100 Device=704
110 !
120 Start_pnt$="00000" ! Sector starting point: 0 deg
130 End_pnt$="10000" ! Sector ending point: 100 deg
140 No_of_scns$="00001" ! No of Scans: 1
150 Incr_gap$="00100" ! Gap between increments: 0.1 deg
160 Velocity$="00667" ! Scan speed: two thirds of max
170 !
180 CLEAR SCREEN
190 CLEAR Device
200 WAIT .5
210 !
220 OUTPUT Device;"HLFe<Aa1<Ae0<Pas";Start_pnt$;"<Pab";End_pnt$;"<J";No_of_scn
s$;"<I";Incr_gap$;"<Va";Velocity$;"<MRH"
230 !
240 WAIT .01
250 OUTPUT Device;"G"
260 !
270 Spoll: S=SPOLL(Device)
280 IF NOT BIT(S,6) THEN GOTO Spoll
290 IF S=92 THEN GOSUB Cust_measure
300 IF S=88 THEN GOTO Complete
310 GOTO Spoll
320 !
330 !
340 *****
350 ! This represents customer measurement routine
360 !
370 Cust_measure: !
380 !
390 Incr_count=Incr_count+1
400 RETURN
410 !
420 *****
430 !
440 Complete: BEEP
450 PRINT "TOTAL INCREMENT COUNT = ";Incr_count
460 END
```



4.6.6 Counting Increments by SRQ using Interrupt

Note: The "SECTOR" scan in the following example will be executed either in "forward" or "reverse" direction, according to the specified direction parameter: "Daf<" or "Dar<" (not included in the program).



AL-4706-3A COMMUNICATION LINK

```
10 *****
20 !* This program executes one SECTOR while counting GPIB increments. *
30 !* The CONTROLLER is instructed to output just the SRQ byte for each *
40 !* increment: the position update is suppressed by the 'Fe<' command. *
50 !*
60 !* The increments are counted using SRQ INTERRUPT method *
70 *****
80 !
90 End_flg=0
100 Incr_count=0
110 Device=704
120 !
130 Start_pnt$="00000" ! Sector starting point: 0 deg
140 End_pnt$="10000" ! Sector ending point: 100 deg
150 No_of_scns$="00001" ! No of Scans: 1
160 Incr_gap$="00100" ! Gap between increments: 0.1 deg
170 Velocity$="00667" ! Scan speed: two thirds of max
180 !
190 CLEAR SCREEN
200 CLEAR Device
210 WAIT .5
220 !
230 ON INTR 7 GOSUB Cust_measure
240 ENABLE INTR 7;2
250 !
260 OUTPUT Device;"HLFe<Aa1<Ae0<Pas";Start_pnt$;"<Pab";End_pnt$;"<J";No_of_scn
s$;"<I";Incr_gap$;"<Va";Velocity$;"<MRH"
270 !
280 WAIT .01
290 OUTPUT Device;"G"
300 !
310 Do_nothing: IF NOT End_flg THEN GOTO Do_nothing
320 PRINT "TOTAL INCREMENT COUNT = ";Incr_count
330 BEEP
340 GOTO Complete
350 !
360 Cust_measure:S=SPOLL(Device)
370 IF S=92 THEN
380 *****
390 ! Here may be situated custommer routine
400 !
410 Incr_count=Incr_count+1
420 !
430 *****
440 END IF
450 IF S=88 THEN End_flg=1
460 !
470 ENABLE INTR 7
480 RETURN
490 !
500 Complete: END
```



APPENDIX A

"MAIN" (V1.3) PROGRAM DOCUMENTATION



A.1 GENERAL

This program activates the AL-4706-3A (V1.6), via GPIB Communication Link.

The program is written in HP BASIC 5.0 and is runnable on HP Series 300 Computers with 12" color monitors. If a 16" monitor is used, ASCII texts will be displaced.

If a non color monitor is used, some of the labels will not come out (as they are written by one color upon another).

A.2 PURPOSE

- Controller and system testing
- Customer Support



A.3 OPERATION INSTRUCTIONS

- * Insert the floppy-disk in drive, load the "MAIN" file and RUN.
- * The program starts by drawing a colorful "ORBIT" logo with two softkeys underneath:

SET-UP

&

CONTIN

- * "SET-UP" softkey is to be used only if CONTROLLER GPIB address and the desired data entering mode in the computer differ from default:

CONTROLLER type AL-4706-3A
CONTROLLER address 704
ENTERING data mode PPOLL

- * "CONTIN" softkey presents to operator a menu of all AL-4706 functions.
- * Function selection will bring a function-dedicated, user-friendly screen, which displays real-time data (position and velocity, system events) as well as operator preselected parameters.
The operator is welcome to change parameters and activate specific functions by means of the appropriate function-keys.

It must be noted that all data presented on screen is feedbacked from the CONTROLLER (this includes preselected parameters display and also error messages.



A.4 HOW DOES IT WORK?

- * The program consists of the following files:

Main core : "MAIN"

Data entering routines: "ENTER DATA"

Polling routines : "MONIT_POLL",
"MONIT_INTR"

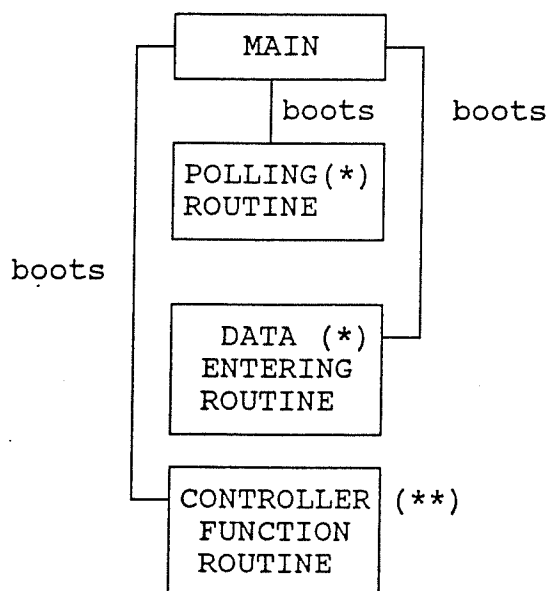
Controller Function Routines : "TRACK" , "RASTER", "SECTOR",
"SLEW-2A", "SLEW" , "STATUS",
: "CONFIG"

Logo File : "LOGO"

Function key preset file : "EMPTY"

All together : 13 files

- * Files resident in memory:

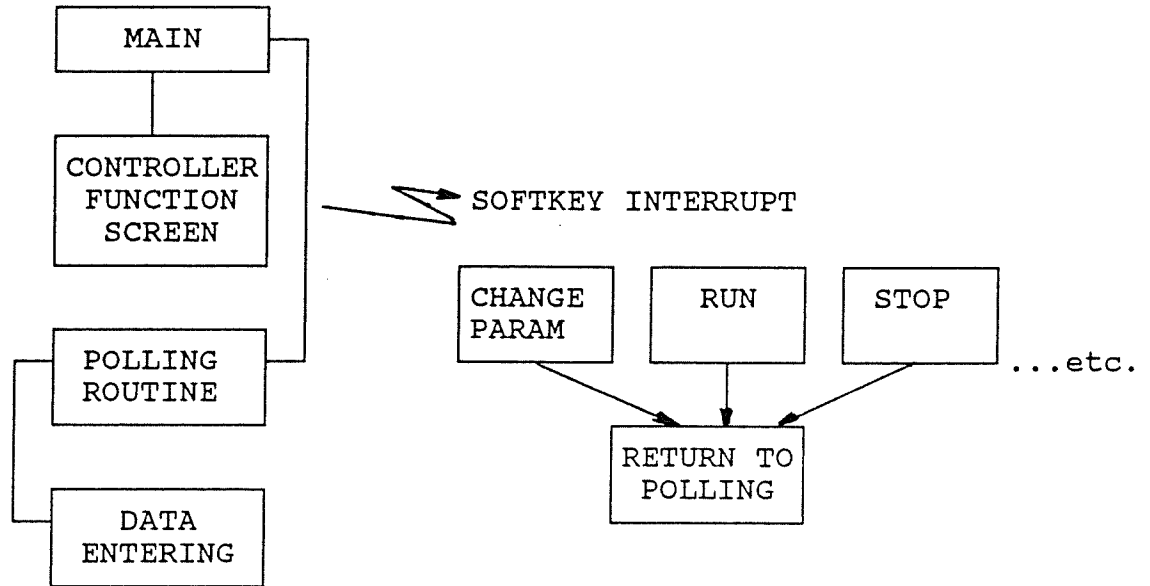


(*) "MONIT_POLL" works with SPOLL or PPOLL
"MONIT_INTR" works with INTR

(**) Whenever a specific CONTROLLER function routine is exited, it is automatically deleted and a new selected function routine is loaded instead.



* Program Flow



* For more detailed information see program lists in conjunction with the AL-4706-3 COMMUNICATION LINK MANUAL.



APPENDIX B

LIST OF ADAPTATIONS FOR:

AL-4806-3A

AL-4906-3A

AL-4901-3A

CONTROLLERS



AL-4706-3A COMMUNICATION LINK

B.1 AL-4806-3A

B.1.1 General

The basic difference between the AL-4706-3A and the AL-4806-3A is that the latter has no EL screen and keyboard. Instead there is a 7-segment display for OPERATING & DISPLAYED axes position and a manual control for one axis out of six.

The Automatic modes (TRACK, RASTER, SECTOR, SLEW) are available only via the Comm Link.

The differences in Comm Link commands regarding to AL-4706-3A are due to variations in software version upgrades.

B.1.2 List of Differences

B.1.2.1 Increments (Para. 3.3.7, 3.3.8) are disabled if the Controller is used as a LOCAL unit for the AL-4706-3B Master Controller (the "fiber optic" option 5).

B.1.2.2 Reading Predefined Direction (Para. 3.4.4).

The response to the D|a|< or D|e|< commands will be:

One letter header ("S"), two zeros, and one letter designating the defined direction:

"r" - for reverse

"f" - for forward

"d" - for auto.

B.1.2.3 Type Parameter (Para. 3.3.10.6) is not applicable.



B.2 AL-4906-3A

B.2.1 General

The basic difference between the AL-4706-3A and the AL-4906-3A is that the latter has no EL screen and keyboard. Instead there is a 7-segment display for the OPERATING axis and manual control for one axis out of six.

The Automatic modes (TRACK, RASTER, SECTOR, SLEW) are available only via the Comm. Link.

The differences in Comm Link commands regarding the AL-4706-3A are due to the fact that there is no indication for a "DISPLAY" axis and also due to variations in software version upgrades.

B.2.2 List of Differences

B.2.2.1 Increments (Para. 3.3.7, 3.3.8) are disabled if the Controller is used as a slave unit for the AL-4706-3B Master Controller (the "fiber optic" option).

B.2.2.2 Reading Predefined Direction (Para. 3.4.4).

The response to the D|a|< or D|e|< commands will be:

One letter header ("S"), two zeros, and one letter designating the defined direction:

"r" - for reverse

"f" - for forward

"d" - for auto.

B.2.2.3 "DISPLAY" axis selection - "En<" command (Para. 3.6, Table 3.2) is not applicable.



AL-4706-3A COMMUNICATION LINK

B.2.2.4 "DISPLAY" axis query - "X2<" command (Para. 3.6, Table 3.2, Para. 3.6.2) - is not applicable.

B.2.2.5 "DISPLAY" axis info in status byte STB1 (Para. 3.6.4) is not applicable.

B.2.2.6 "TYPE" Parameter (para. 3.3.10.6) is not applicable.

B.2.3 Software Examples

All software examples in Chapter 4 are runnable. However, the operator must take into consideration the differences and limitations as described herein.

If "MAIN" program is used, its SET-UP must be updated to the appropriate Controller type.



B.3 AL-4901-3A

B.3.1 General

The basic difference between the AL-4706-3A and the AL-4901-3A is that the latter has no EL screen and keyboard. Instead there is a 7-segment display for the position readout and a manual control for only one axis.

The Automatic modes (TRACK, RASTER, SECTOR, SLEW) are available only via Comm Link.

The differences in Comm Link commands regarding the AL-4706-3A are due to the fact that there is no axis selection (only one axis operation) and also due to variations in software version upgrades.

B.3.2 List of Differences

B.3.2.1 Increments (Para. 3.3.7, 3.3.8) are disabled if the Controller is used as a slave unit for the AL-4706-3B Master Controller (the "fiber optic" option).

B.3.2.2 Reading Predefined Direction (Para. 3.4.4).

The response to the D|a|< or D|e|< commands will be:

One letter header ("S"), two zeros, and one letter designating the defined direction:

"r" - for reverse

"f" - for forward

"d" - for auto.

B.3.2.3 "DISPLAY" axis selection - "En<" command (Para. 3.6, Table 3.2) is not applicable.



AL-4706-3A COMMUNICATION LINK

- B.3.2.4 "DISPLAY" axis query - "X2<" command (Para. 3.6, Table 3.2, Para. 3.6.2) - is not applicable.
- B.3.2.5 "DISPLAY" axis info in status byte STB1 (Para. 3.6.4) is not applicable.
- B.3.2.6 Boundary position (Para. 3.3.1 & 3.4.1)
The following commands are not applicable:
"Pes" - Starting position of step
"Peb" - Ending position of step
"Pet" - Target position of second TRACK axis
- B.3.2.7 Axis selection (Para. 3.3.2 & 3.4.2) is not applicable.
- B.3.2.8 Velocity (Para. 3.3.3 & 3.4.3)
"Ve" - is the velocity of moving towards the starting point in SECTOR mode.
- B.3.2.9 Direction (Para. 3.3.4 & 3.4.4)
"De" - Command is not applicable.
- B.3.2.10 Mode of Motion (Para. 3.3.5)
M|R - Sector Scan A
M|N - Sector Scan B
- B.3.2.11 Step Value or Number of Scans (Para. 3.3.6, 3.4.6)
"J" command means only: Number of Scans.
- B.3.2.12 Configuration Parameters (Para. 3.3.10, 3.4.7)
Have to be prefixed by an "A1<" command.
- B.3.2.13 Type Parameter (para. 3.3.10.6) is not applicable.
- B.3.3 Software Examples:

All software examples in Chapter 4 are runnable. However, the operator must take into consideration the differences and limitations as described herein.

If "MAIN" program is used, its SET-UP must be updated to the appropriate Controller type.



AL-4706-3A COMMUNICATION LINK

- B.3.2.4 "DISPLAY" axis query - "X2<" command (Para. 3.6, Table 3.2, Para. 3.6.2) - is not applicable.
- B.3.2.5 "DISPLAY" axis info in status byte STB1 (Para. 3.6.4) is not applicable.
- B.3.2.6 Boundary position (Para. 3.3.1 & 3.4.1)
The following commands are not applicable:
"Pes" - Starting position of step
"Peb" - Ending position of step
"Pet" - Target position of second TRACK axis
- B.3.2.7 Axis selection (Para. 3.3.2 & 3.4.2) is not applicable.
- B.3.2.8 Velocity (Para. 3.3.3 & 3.4.3)
"Ve" - is the velocity of moving towards the starting point in SECTOR mode.
- B.3.2.9 Direction (Para. 3.3.4 & 3.4.4)
"De" - Command is not applicable.
- B.3.2.10 Mode of Motion (Para. 3.3.5)
M|R - Sector Scan A
M|N - Sector Scan B
- B.3.2.11 Step Value or Number of Scans (Para. 3.3.6, 3.4.6)
"J" command means only: Number of Scans.
- B.3.2.12 Configuration Parameters (Para. 3.3.10, 3.4.7)
Have to be prefixed by an "A1<" command.
- B.3.2.13 Type Parameter (para. 3.3.10.6) is not applicable.
- B.3.3 Software Examples:

All software examples in Chapter 4 are runnable. However, the operator must take into consideration the differences and limitations as described herein.

If "MAIN" program is used, its SET-UP must be updated to the appropriate Controller type.



AL-4706-3A COMMUNICATION LINK

- B.3.2.4 "DISPLAY" axis query - "X2<" command (Para. 3.6, Table 3.2, Para. 3.6.2) - is not applicable.
- B.3.2.5 "DISPLAY" axis info in status byte STB1 (Para. 3.6.4) is not applicable.
- B.3.2.6 Boundary position (Para. 3.3.1 & 3.4.1)
The following commands are not applicable:
"Pes" - Starting position of step
"Peb" - Ending position of step
"Pet" - Target position of second TRACK axis
- B.3.2.7 Axis selection (Para. 3.3.2 & 3.4.2) is not applicable.
- B.3.2.8 Velocity (Para. 3.3.3 & 3.4.3)
"Ve" - is the velocity of moving towards the starting point in SECTOR mode.
- B.3.2.9 Direction (Para. 3.3.4 & 3.4.4)
"De" - Command is not applicable.
- B.3.2.10 Mode of Motion (Para. 3.3.5)
M|R - Sector Scan A
M|N - Sector Scan B
- B.3.2.11 Step Value or Number of Scans (Para. 3.3.6, 3.4.6)
"J" command means only: Number of Scans.
- B.3.2.12 Configuration Parameters (Para. 3.3.10, 3.4.7)
Have to be prefixed by an "A1<" command.
- B.3.2.13 Type Parameter (para. 3.3.10.6) is not applicable.
- B.3.3 Software Examples:

All software examples in Chapter 4 are runnable. However, the operator must take into consideration the differences and limitations as described herein.

If "MAIN" program is used, its SET-UP must be updated to the appropriate Controller type.



AL-4706-3A COMMUNICATION LINK

- B.3.2.4 "DISPLAY" axis query - "X2<" command (Para. 3.6, Table 3.2, Para. 3.6.2) - is not applicable.
- B.3.2.5 "DISPLAY" axis info in status byte STB1 (Para. 3.6.4) is not applicable.
- B.3.2.6 Boundary position (Para. 3.3.1 & 3.4.1)
The following commands are not applicable:
"Pes" - Starting position of step
"Peb" - Ending position of step
"Pet" - Target position of second TRACK axis
- B.3.2.7 Axis selection (Para. 3.3.2 & 3.4.2) is not applicable.
- B.3.2.8 Velocity (Para. 3.3.3 & 3.4.3)
"Ve" - is the velocity of moving towards the starting point in SECTOR mode.
- B.3.2.9 Direction (Para. 3.3.4 & 3.4.4)
"De" - Command is not applicable.
- B.3.2.10 Mode of Motion (Para. 3.3.5)
M|R - Sector Scan A
M|N - Sector Scan B
- B.3.2.11 Step Value or Number of Scans (Para. 3.3.6, 3.4.6)
"J" command means only: Number of Scans.
- B.3.2.12 Configuration Parameters (Para. 3.3.10, 3.4.7)
Have to be prefixed by an "A1<" command.
- B.3.2.13 Type Parameter (para. 3.3.10.6) is not applicable.
- B.3.3 Software Examples:

All software examples in Chapter 4 are runnable. However, the operator must take into consideration the differences and limitations as described herein.

If "MAIN" program is used, its SET-UP must be updated to the appropriate Controller type.